Technická univerzita v Košiciach
Fakulta elektrotechniky a informatiky

Mikro-tendre Studentskej rady - on-chain denník malých nákupov

Bakalárska práca

2025 **Kostiantyn Ostrovskyi**

Technická univerzita v Košiciach
Fakulta elektrotechniky a informatiky

Mikro-tendre Studentskej rady - on-chain denník malých nákupov

Bakalárska práca

Študijný program:	Informatika
Študijný odbor:	Informatika
Školiteľ:	Ing. Daniel Šolc
Školiace pracovisko:	Katedra počítačov a informatiky (KPI)

Košice 2025 **Kostiantyn Ostrovskyi**

Abstrakt v slovenčine

Táto práca predstavuje riešenie problému malých nákupov študentskej rady prostredníctvom vytvárania tendrov. Po analýze riešení konkurentov, konkrétne tradičných tendrových platforiem a systémov automatického riadenia financií, je zrejmé, že väčšina z nich používa príliš veľa nepotrebných nástrojov, ktoré sú príliš drahé a nevhodné pre študentské organizácie. Preto sa v tejto práci bude skúmať inovatívne riešenie — vytvorenie decentralizovanej aplikácie (dApp) na báze technológie blockchain.

Kľúčové slová v slovenčine

LaTeX, programovanie, mikrotendre, študentská rada, blockchain, smart kontrakty, IPFS, transparentnosť nákupov, kvorum, decentralizácia, TUKE

Abstrakt v angličtine

This thesis presents a solution to the problem of small purchases by student councils through the creation of tenders. After analyzing competitors' solutions, specifically traditional tender platforms and automated financial management systems, it can be concluded that most of them use too many unnecessary tools that are too expensive and unsuitable for student organizations. Therefore, this thesis will explore an innovative solution — the creation of a decentralized application (dApp) based on blockchain technology.

Kľúčové slová v angličtine

LaTeX, programming, micro-tenders, student council, blockchain, smart contracts, IPFS, procurement transparency, quorum, decentralization, TUKE

Bibliografická citácia

OSTROVSKYI, Kostiantyn. *Mikro-tendre Studentskej rady - on-chain denník malých nákupov*. Košice: Technická univerzita v Košiciach, Fakulta elektrotechniky a informatiky, 2025. 47 s. Vedúci práce: Ing. Daniel Šolc

Na túto stranu vložte zadávací list. To urobíte nasledovne:

1. Zadávací list naskenujte v odtieňoch sivej (čiernobielo) na 200 až 300 DPI.
2. Sken uložte do priečinku `figures/`.
3. V súbore `metadata.tex` zadajte umiestnenie súboru pomocou príkazu `\thesisspec{figures/zadavaci.list}`

Do jednej kópie práce však nezabudnite vložiť originál zadávacieho listu!

Čestné vyhlásenie

Vyhlasujem, že som záverečnú prácu vypracoval samostatne s použitím uvedenej odbornej literatúry.

Poďakovanie

Na tomto mieste by som sa rád poďakoval svojmu vedúcemu práce za jeho čas a odborné vedenie počas riešenia mojej záverečnej práce. Rovnako by som sa rád poďakoval svojim rodičom a priateľom za ich podporu a povzbudzovanie počas celého môjho štúdia.

Obsah

1 Úvod **1**

2 A little bit background **4**

2.1 Existujúce systémy tendrov 4

2.2 Tender system evaluation 5

2.2.1 Security properties 5

2.3 Blockchain technology 5

2.3.1 General Working Sequence of Blockchain 6

2.3.2 Transaction 7

2.3.3 Composition of a Block 7

2.3.4 Blockchain Platforms 8

2.3.5 Smart-contracts 10

2.3.6 Hlasovanie na blockchaine 10

2.4 IPFS 10

2.4.1 Principles of IPFS operation 11

2.4.2 Why IPFS for documents? 12

2.4.3 Alternative 13

2.4.4 Integration with Blockain 14

3 Requirements specification **16**

3.1 Functional requirements 16

3.1.1 Functional requirements for student council members 16

3.1.2 Functional requirements for suppliers 17

3.1.3 Functional requirements related to blockchain technology . 19

3.2 Non-functional requirements 19

3.3 Use cases 20

4 Solution design **21**

4.1 Overall system architecture 21

4.2 Smart contract design 22

4.2.1 Contract architecture 22
4.2.2 Data model 23
4.2.3 Tender lifecycle 23
4.3 IPFS integration design 25
4.3.1 Data processing workflow 25
4.4 User interface design (UI/UX) 25
4.4.1 Wireframes 26
4.4.2 UX principles 27
4.5 User roles and permission model 27

5 Implementation **28**
5.1 Development environment and technologies 28
5.2 Smart contract implementation 28
5.2.1 Contract structure and state variables overview 29
5.2.2 Constructor and initialization 30
5.2.3 Data structures 30
5.2.4 Access control 31
5.2.5 Events 32
5.2.6 Key contract functions 32
5.2.7 Transition and administrative functions 36
5.2.8 Helper functions and edge cases 36
5.2.9 Security aspects of the implementation 37
5.2.10 Gas cost optimization 37
5.2.11 Súhrn rozsahu implementácie kontraktu 38
5.3 Implementácia klientskej aplikácie (Frontend) 38
5.3.1 Štruktúra aplikácie a vstupný bod (App.jsx) 38
5.3.2 Hlavné komponenty a stránky 39
5.3.3 Pripojenie peňaženky a inštancia kontraktu 40
5.3.4 Integrácia s IPFS (Pinata) 40
5.3.5 Súhrn frontendu 41

6 Vyhodnotenie **42**

7 Záver **43**

Zoznam použitej literatúry **44**

Zoznam príloh **48**

Zoznam obrázkov

2.1 Príklad architektúry blockchainového systému [2] 6
2.2 The structure of a Blockchain[5] 7
2.3 Difference beetween IPFS and HTTP[15] 11
3.1 Diagram prípadov použitia pre systém tendrov 20
4.1 Celková architektúra systému pre správu mikro-tendrov 21
4.2 Logická štruktúra smart kontraktov 23
4.3 Stavový diagram životného cyklu tendra 24
4.4 Sekvenčný diagram interakcie s IPFS 25
4.5 Návrh hlavného panela (Dashboard) 26
4.6 Návrh formulára pre vytvorenie tendra 26

Zoznam tabuliek

2.1 Porovnanie elektronických verejných obstarávacích systémov 5
2.2 Hlavné oblasti použitia rôznych blockchainových systémov 8
2.3 Comparative Overview of Blockchain Platforms (2025) 9
2.4 Porovnanie protokolov HTTP a IPFS 11
2.5 Porovnanie protokolov HTTP, IPFS, Filecoin a Arweave 14

Zoznam výpisov

5.1 Štruktúra Tender v kontrakte MicroTender.sol 30
5.2 Štruktúra Bid pre ponuku dodávateľa 31
5.3 Modifikátor onlyMember pre prístup členov rady 32
5.4 Vybraté deklarácie eventov v MicroTender.sol 32
5.5 Jadro funkcie createTender (validácia a zápis) 33
5.6 Validácie a zápis v submitBid 34
5.7 Záznam hlasu v castVote 34
5.8 Určenie víťaza v finalizeTender 35
5.9 Zjednodusena struktura App.jsx s routerom a Web3 kontextom 39

1 Úvod

Študentské rady vo vzdelávacích inštitúciách pravidelne nakupujú rôzne položky na podujatia, zásoby a iné účely. Sumy takýchto nákupov sú zvyčajne malé, ale kľúčovým aspektom, ktorý treba zvážiť, zostáva potreba systematického prístupu k účtovníctvu a kontrole. To často vedie k problémom, pretože je ťažké, ak nie nemožné, overiť transparentnosť takýchto transakcií. Za zmienku tiež stojí, že značný čas strávený rozhodovaním zostáva významnou výzvou.

Vzhľadom na všetky uvedené skutočnosti sa zdá pravdepodobné, že dobrým riešením by bolo použitie systému verejného obstarávania, ktorý dokáže tento problém efektívne riešiť. Toto skutočne môže byť dobré riešenie, ale rád by som navrhol zvážiť technicky pokročilejšiu verziu systému verejného obstarávania založeného na technológii blockchain s využitím inteligentných zmlúv vo forme decentralizovanej aplikácie (dApp). Medzi výhody použitia technológie blockchain patrí transparentnosť a nemennosť údajov, decentralizácia, sledovanie transakcií, automatizácia prostredníctvom inteligentných zmlúv, dôvera bez sprostredkovateľov a predovšetkým bezpečnosť. Tieto vlastnosti sú nevyhnutné, pokiaľ ide o finančné transakcie, ako v našom prípade.

Po analýze konkurenčných riešení, konkrétne konvenčných platforiem pre verejné obstarávanie a automatizovaných systémov finančného riadenia, vidíme, že väčšina z nich používa príliš veľa nepotrebných nástrojov, je príliš drahá a nevhodná pre študentské organizácie. Hoci blockchain nevyrieši každý problém a nie je svätým grálom, táto technológia ponúka nové možnosti využitia, ktoré nám určite uľahčia úlohu. Cieľom tejto práce je preto vytvoriť inovatívnu decentralizovanú aplikáciu, ktorá zabezpečí bezpečnosť, dôveru a efektívnosť v mikro-tendroch.

Práca sa zameriava nielen na implementáciu, ale aj na porovnanie jej výhod oproti konkurencii v reálnom prostredí, konkrétne pri použití študentskou radou na malé nákupy. Výsledná decentralizovaná aplikácia v konečnom dôsledku demonštruje, ako možno technológiu blockchain využiť na zlepšenie transparentnosti a efektívnosti finančného riadenia.

Formulácia úlohy

Študentské rady zohrávajú dôležitú úlohu vo vnútornom živote univerzít. Pripravujú a organizujú podujatia, nakupujú materiály a ďalšie aktivity, z ktorých väčšina si vyžaduje finančné výdavky. Hoci mnohé z týchto výdavkov sú malé, ich objem je značný a účtovníctvo sa často prehliada. To vedie k množstvu problémov, od straty údajov a neefektívnych kontrol až po nedostatok transparentnosti vo finančných transakciách.

Väčšina správ o menších nákupoch sa ukladá v excelovských tabuľkách, čo neposkytuje potrebnú úroveň spoľahlivosti a transparentnosti, pretože ktokoľvek môže údaje sfalšovať, a to aj náhodným pochybením.

Technologický pokrok ponúka príležitosti na automatizáciu takýchto procesov. Vytvorenie online denníka menších nákupov môže výrazne zjednodušiť prácu študentskej rady tým, že poskytuje užívateľsky prívetivé rozhranie na vedenie záznamov, ukladanie informácií a generovanie správ.

Technológia blockchain, ktorá zvyšuje dôveru a transparentnosť vo finančných transakciách, je obzvlášť účinná pri riešení takýchto problémov. Decentralizovaná štruktúra zabezpečuje, že údaje nemožno meniť a všetky záznamy o nákupoch sú overiteľné. Týmto spôsobom pomáha udržiavať čestnosť a transparentnosť so študentmi pri rozdeľovaní finančných prostriedkov.

Hlavným cieľom tejto bakalárskej práce je vyvinúť prototyp decentralizovanej aplikácie (DApp) na vytváranie a vykonávanie mikrotendrov študentskou radou TUKE. Táto aplikácia zabezpečí automatizáciu a transparentnosť procesu obstarávania.

Systém umožní členom študentskej rady iniciovať požiadavky na obstarávanie, zhromažďovať ponuky od dodávateľov, vykonávať schvaľovanie kvóra a spracovávať platby pomocou inteligentných zmlúv na blockchaine Polygon. Všetky dokumenty a záznamy o transakciách budú uložené v decentralizovanom úložisku IPFS, čím sa zabezpečí ich dostupnosť pre každého študenta a ochrana pred úpravami.

Na dosiahnutie cieľa tejto práce budú implementované nasledujúce kroky:

1. Analýza finančných procesov. Študovať procesy obstarávania prostredníctvom tendrov, vrátane tých, ktoré vykonávajú študentské organizácie, a identifikovať kľúčové problémy, ktoré je potrebné riešiť.

2. Preskúmať existujúce riešenia. Porovnať výhody a nevýhody systémov tendrov, vrátane tých, ktoré nevyužívajú technológiu blockchain, a zhodnotiť ich vhodnosť pre študentské organizácie.

3. Analýza existujúcich technológií (blockchain, IPFS, DApp). Štúdium princípov blockchainu a inteligentných zmlúv, ako aj výber správnej platformy pre danú

prácu.

4. Vývoj a implementácia architektúry systému. Návrh architektúry systému, modelu databázy, API a UI/UX. Vývoj inteligentných zmlúv, backendových a frontendových aplikácií.

5. Testovanie. Testovanie funkčnosti, bezpečnosti a výkonu systému v praxi.

2 A little bit background

V tejto časti analyzujeme a hodnotíme tri existujúce systémy tendrov (Prozorro, TenderNed, EKS), poskytujeme stručný prehľad hlavných konceptov použitých v našej práci a diskutujeme o súvisiacej literatúre.

2.1 Existujúce systémy tendrov

Existuje mnoho riešení určených pre účtovníctvo obstarávania a finančné riadenie. Väčšina z nich je však určená pre korporátny alebo vládny sektor a nie je vhodná pre potreby menších organizácií vrátane študentských rád. Analýzou a štúdiom týchto systémov je však možné identifikovať niektoré princípy, na ktoré sa možno odvolávať pri vývoji vlastného produktu.

Ako príklad si vezmime tri systémy verejných zákaziek v rôznych krajinách a skúsme ich porovnať.

- Prozorro [1]
- TenderNed (Holandsko)[2]
- Elektronický kontraktačný systém (Slovensko)[3]

Medzi ich výhody patrí vysoká úroveň transparentnosti a právnej ochrany, ako aj podávanie správ a audit, keďže tendre sa vykonávajú na vládnej úrovni. Hlavnou nevýhodou však je, že všetky tieto systémy tendrov sú určené pre veľké podniky a korporácie.

Na základe uvedenej tabuľky2.1 je potrebné zohľadniť všetky pozitívne aspekty existujúcich tendrov (rýchlosť, transparentnosť, spoľahlivosť) a opraviť tie negatívne (vytvoriť jednoduché rozhranie postačujúce pre malé nákupy určené len pre študentskú radu).

[1] https://prozorro.gov.ua/en

[2] https://www.tenderned.nl/cms/en/english/tenderned-dutch-governments-online--tendering-system

[3] https://www.eks.sk/

Tabuľka 2.1: Porovnanie elektronických verejných obstarávacích systémov

Systém	Krajina	Opis	Výhody	Nevýhody
Prozorro	Ukrajina	Národný systém verejného obstarávania navrhnutý na boj proti korupcii	Transparentnosť, otvorené dáta, verejná kontrola	Komplexný pre malé nákupy, potreba školenia používateľov
EKS	Slovensko	Elektronický kontraktačný systém pre malé a stredné verejné obstarávania	Jednoduchosť, rýchlosť, transparentnosť	Neobľúbenosť medzi podnikateľmi, zastarané rozhranie
TenderNed	Holandsko	Štátna platforma pre všetky verejné tendre v krajine	Centralizovaný prístup, otvorené dáta, vysoká spoľahlivosť	Jazyková bariéra, zložitosť pre menšie organizácie

2.2 Tender system evaluation

Systém výberových konaní musí byť spoľahlivý a vzbudzovať dôveru medzi používateľmi. Na dosiahnutie tohto cieľa musí byť transparentný, a práve tu prichádza na rad blockchain. Musíme tiež zvážiť bezpečnosť a efektívnosť hlasovacích systémov a platobných systémov využívajúcich smart kontraktov.

2.2.1 Security properties

Náš systém výberových konaní musí spĺňať nasledujúce podmienky:

1. Auditabilita: všetky operácie – od podania žiadosti až po uzavretie zmluvy musia byť auditovateľné.
2. Transparentnosť: všetky informácie o výberovom konaní musia byť viditeľné pre všetkých študentov, dokonca aj pre tých, ktorí nie sú členmi študentskej rady.
3. Nezmeniteľnosť: Po zverejnení údajov na blockchaine ich nemožno zmeniť ani odstrániť.

2.3 Blockchain technology

Blockchain je distribuovaná digitálna účtovná kniha, ktorá ukladá dáta akéhokoľvek typu. Blockchain dokáže zaznamenávať informácie o transakciách s kryptomenami, vlastníctve NFT alebo inteligentných zmluvách DeFi. Zatiaľ čo akákoľvek tradičná databáza dokáže ukladať takéto informácie, blockchain je jedinečný v tom, že je úplne decentralizovaný. Namiesto toho, aby bol uložený na jednom mieste

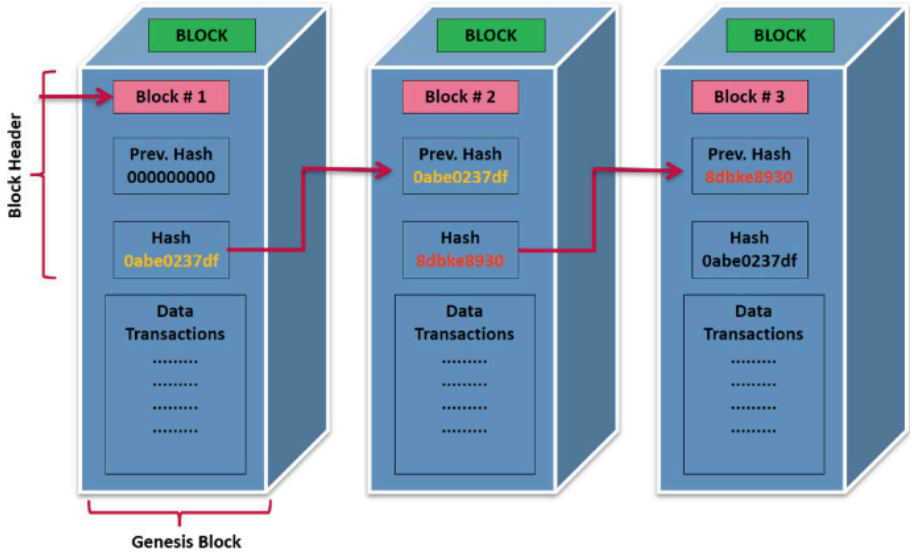
centralizovaným správcom – napríklad v tabuľke programu Excel alebo v bankovej databáze – viacero identických kópií blockchainovej databázy je uložených na viacerých počítačoch roztrúsených po sieti. Tieto jednotlivé počítače sa nazývajú uzly. [1]

Vďaka tejto vlastnosti rieši používanie blockchainu problém dôvery, pretože žiadna osoba nemôže meniť žiadne dáta pre svoje vlastné účely.

Bude jasné, do ktorej peňaženky[4] note boli peniaze prevedené, kedy, od koho atď.

2.3.1 General Working Sequence of Blockchain

Obrázok 2.1 zobrazuje obsah bloku v blockchainovej sieti, pričom konkrétne ukazuje hlavičku bloku a blok genesis.



Obrázok 2.1: Príklad architektúry blockchainového systému [2]

Nižšie rozoberieme všeobecnú pracovnú postupnosť blockchainu:

1.Používateľ(i) blockchainu vytvorí(jú) účet(y).

2.Používateľ blockchainu vytvorí transakciu (Tx).

3.Transakcie sú podpísané.

4.Transakcia je vysielaná na overovacie uzly.

5.Transakcia je validovaná.

6.Transakcie sú zhromaždené v poole.

7.Uzol(y) ťažobného(ých) uzla(ov) zhromaždí(jú) viacero Tx a vytvorí blok.

8.Uzol(y) ťažobného(ých) uzla(ov) vykonáva(jú) ťažbu.

[4]Crypto-wallet

9.Vykoná sa validácia bloku.

10.Úspešný uzol ťažobného pridá blok do blockchainu.

11.Informácie o pridanom bloku sú vysielané do blockchainovej siete.

12.Uzol blockchainu pridá vysielaný blok do svojej lokálnej kópie blockchainu – potvrdenie bloku.

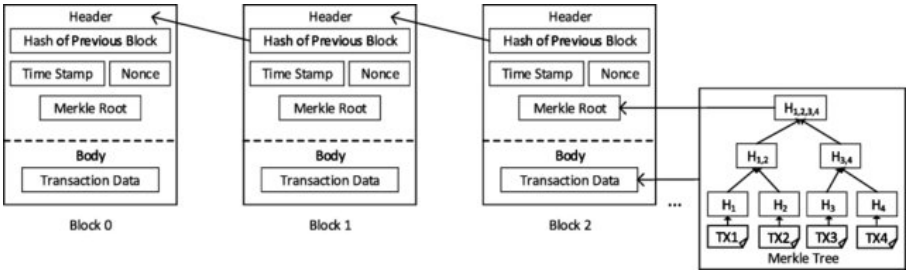
13.Blok sa stáva súčasťou globálneho blockchainu – blok je publikovaný.

2.3.2 Transaction

Blockchain dokáže spracovať rôzne formy informácií, ako napríklad kryptomenu, digitálne aktíva, informácie o kupujúcom alebo predávajúcom a akékoľvek ďalšie informácie, ako transakcie[3]. To závisí od aplikácie konkrétneho blockchainu. Transakcie môžu tiež obsahovať súbor informácií. Používateľ môže vytvoriť jednu alebo viac transakcií. Vytvorenie týchto transakcií môže od používateľa blockchainu vyžadovať rôzne informácie a môžu sa líšiť v závislosti od implementácie blockchainu. Transakcia musí obsahovať všeobecné informácie, ako napríklad prenášané aktívum. Vyžadujú sa aj informácie o odosielateľovi a príjemcovi. Okrem toho musia byť zahrnuté informácie o vysporiadaní transakcie[2]. Po vytvorení transakcie je potrebné ju podpísať a následne overiť. Po podpísaní sa transakcie vysielajú do blockchainovej siete, kde ich overovacie uzly overia a pridajú do bloku. Ťažobné uzly potom overia bloky pomocou konsenzuálneho algoritmu a ak je všetko v poriadku, blok sa považuje za úspešne vyťažený. Keď sa blok považuje za úspešne vyťažený, jeho informácie už nemožno meniť. Akékoľvek zmeny je možné vykonať iba pridaním nových blokov[4].

2.3.3 Composition of a Block

Blok sa skladá z niekoľkých vecí. Obrázok zobrazuje komponenty bloku, ktoré popíšeme nižšie:



Obrázok 2.2: The structure of a Blockchain[5]

Blok sa skladá z hlavičky a tela, kde hlavička obsahuje hash predchádzajúceho

bloku, časovú pečiatku, Nonce a Merkleho koreň. Merkleho koreň je hash koreňa Merkleho stromu, ktorý je uložený v tele bloku. Transakciu označujeme ako TX a ako príklad na ilustráciu štruktúry Merkleho stromu berieme tretí blok, ktorý obsahuje iba štyri transakcie.[5]

2.3.4 Blockchain Platforms

Tabuľka zobrazuje aplikáciu blockchainu a rôzne blockchainové systémy, ktoré takéto aplikácie podporujú.

Tabuľka 2.2: Hlavné oblasti použitia rôznych blockchainových systémov

Application of blockchain	**Blockchain systems**
Asset Management and Tracking	Corda, BigchainDB, Multichain, VeChain, Hyperledger Fabric
Own assets in terms of Token	Stellar, IOTA, Tezos, Algorand, Cardano
Cryptocurrency	Bitcoin, Litecoin, Dogecoin, Monero, Dash
Smart Contract-based new Business Logic	Parity, Ethereum, Quorum, Solana, Avalanche, Binance Smart Chain, Polygon
Decentralized Storage	Filecoin, Arweave, Storj
Identity Management	Sovrin, uPort, Civic
Decentralized Finance (DeFi)	Ethereum, Avalanche, Solana, Terra
Interoperability / Cross-chain	Polkadot, Cosmos
Privacy-focused Transactions	Zcash, Monero, Secret Network

Ako vidíme, existuje obrovské množstvo blockchainov používaných v rôznych oblastiach. Musíme si vybrať jeden, ktorý použijeme pre náš systém platieb. Prioritou je, aby tento blockchain podporoval inteligentné zmluvy. Na tento účel sa pozrieme na ďalšiu, podrobnejšiu tabuľku 2.3 porovnávajúcu najpopulárnejšie blockchainy, ktoré podporujú inteligentné zmluvy[6].

Gas Fees

Poplatky za transakciu sú veľmi dôležitým kritériom! Nikto nechce platiť 1 – 2 doláre za transakciu, najmä ak počas procesu vytvárania ponuky môžu byť takéto transakcie dve alebo viac. Preto by sme si mali vybrať najlacnejšiu, čo je v tomto prípade voľba medzi Solanou a Polygonom.

Tabuľka 2.3: Comparative Overview of Blockchain Platforms (2025)

Criterion	Ethereum	Polygon	BSC	Solana
Launch Year	2015	2017 (as Matic)	2020	2020
Type	Layer 1	Layer 2 / Sidechain	Layer 1	Layer 1
Consensus Mechanism	Proof of Stake (PoS)	Proof of Stake (PoS)	Proof of Staked Authority (PoSA)	Proof of Stake (PoS)
Real TPS	~25	~75	~300	~720
Average Fee (2025)	~$0.1285	~$0.003	~$0.02	~$0.008
EVM Compatibility	Yes (native)	Yes (full)	Yes (full)	No (custom VM)
Smart Contract Languages	Solidity, Vyper	Solidity, Vyper	Solidity	Rust, C, C++
Native Token	ETH	MATIC (POL)	BNB	SOL
Block Time	~12s	~2s	~0.75s	~0.4s
Daily Active Wallets	~530K	~1.7M	~4M	~2M
Decentralization Level(Count of validators)	~973.0K	~100	~45	~800
Number of dApps[7]	5.000+	2,500+	6.000+	700+
Ecosystem Size	Largest	Large	Large	Growing

Source: Data compiled from [8, 9, 10].

Speed and performance

Tu je situácia takmer rovnaká pre všetky blockchainy, body počítame všetkým, okrem Ethereum, ktoré je príliš pomalé na naše účely.

Solidity support (EVM Compatibility)

Najpopulárnejším jazykom na písanie inteligentných zmlúv je Solidity. Je veľmi užívateľsky prívetivý a široký a má veľa knižníc, takže v ňom budeme písať všetok kód pre náš systém výberových konaní. Solana bohužiaľ nie je vhodná, pretože potrebujeme iný programovací jazyk, dokonca viac ako jeden, ako napríklad Rust a C++[11]. To má svoje výhody, ako je lepšia bezpečnosť a výkon, ale medzi jeho hlavné nevýhody patrí zložitejšia krivka učenia a menej príležitostí na použitie v ekosystéme Web 3.0.

Decentralization and Security

Ethereum má najviac validátorov[12], čo zabezpečuje najlepšiu decentralizáciu zo všetkých konkurentov na tomto zozname. Na druhom mieste je Solana, ktorá má výrazne menej validátorov, ale stále sa môže pochváliť veľmi vysokou úrovňou decentralizácie. Polygon a BSC sú na poslednom mieste, ale to neznamená, že sú menej decentralizované; jednoducho majú menej validátorov potvrdzujúcich transakcie v sieti v porovnaní s prvým z nich.

Winner

Na základe uvedených kritérií je Polygon[5] pre naše účely perfektnou voľbou vďaka nízkym nákladom na transakcie a podpore inteligentných zmlúv a Solidity. Tabuľka ukazuje, že Polygon je široko používaný, s približne 1.7 miliónmi peňaženiek používaných denne, a veľmi veľký počet dApp si vybral Polygon pre vývoj. Ethereum, ktorý je dobre otestovaný a používa ho prevažná väčšina dApp, by mohol byť dobrou alternatívou, ale trpí vo veľmi dôležitej oblasti: transakčných poplatkoch.[6] Mohla by nastať situácia, keď by sa ponuka 20 dolárov mohla ukázať ako nerentabilná, pretože transakcie by stáli 25 – 30 dolárov.

2.3.5 Smart-contracts

Smart contracts sú digitálne zmluvy uložené v blockchaine, ktoré sa automaticky vykonajú, keď sú splnené vopred určené podmienky[13]. Inteligentné zmluvy fungujú ako elektronické dohody. V našom prípade budú hrať neuveriteľne dôležitú úlohu vo funkčnosti našich ponúk. Napríklad s ich pomocou môžeme vytvárať roly, pričom každá bude mať prístup ku konkrétnej funkcii. Tieto funkcie budeme tiež písať pomocou inteligentných zmlúv.

2.3.6 Hlasovanie na blockchaine

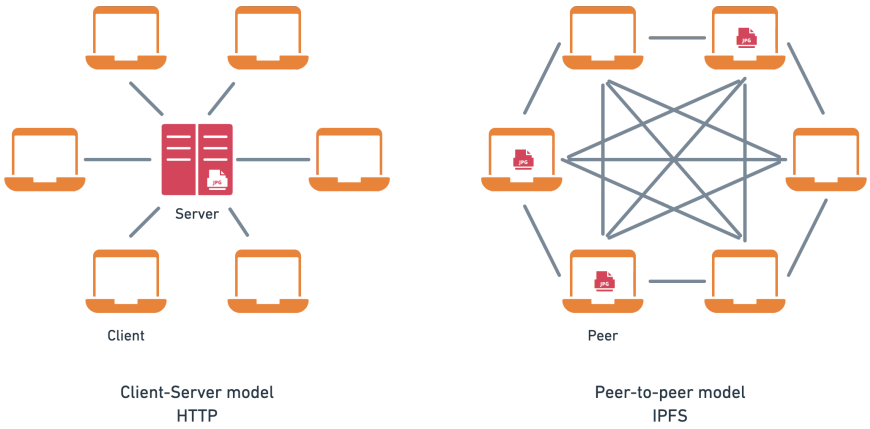
Blockchain sa v modernom svete už hojne používa na hlasovanie, pretože rieši problémy s integritou výsledkov a transparentnosťou hlasovania. V našom prípade bude proces hlasovania implementovaný s cieľom vybrať najlepšiu ponuku od predajcov. Blockchain sa tiež často spája s anonymitou, takže máme možnosť vytvoriť anonymné hlasovanie, hoci to v našom prípade nie je dôležité.

2.4 IPFS

IPFS (Inter-Planetary File System) je distribuovaný súborový systém typu peer--to-peer, ktorý sa používa predovšetkým pre dáta, ktoré nemožno uložiť na blockchain. Na rozdiel od HTTP, štandardnejšieho protokolu na zdieľanie súborov, má IPFS špeciálne vlastnosti, vďaka ktorým je ideálny pre decentralizovaný dátový model, ktorý funguje v tandeme s blockchainmi[14].

[5] https://polygon.technology/

[6] Pri extrémnom zaťažení siete môžu byť poplatky neuveriteľne vysoké.



Obrázok 2.3: Difference beetween IPFS and HTTP[15]

V tradičných webových aplikáciách sa na prenos a ukladanie dát používa protokol HTTP. V decentralizovaných systémoch, najmä v spojení s blockchain technológiami, sa však čoraz častejšie využíva IPFS, ktorý rieši viaceré obmedzenia centralizovaných riešení (Tab. 2.4).

Tabuľka 2.4: Porovnanie protokolov HTTP a IPFS

Kritérium	**HTTP**	**IPFS**
Architektúra	Centralizovaná klient–server	Decentralizovaná peer-to-peer
Adresovanie obsahu	Podľa umiestnenia (URL, server)	Podľa obsahu (hash súboru – CID)
Odolnosť voči cenzúre	Nízka – server môže byť vypnutý alebo blokovaný	Vysoká – obsah je distribuovaný medzi uzlami
Integrita dát	Závisí od dôvery k serveru	Kryptograficky garantovaná hashom
Dostupnosť	Závislá od dostupnosti servera	Závislá od počtu dostupných uzlov
Škálovateľnosť	Obmedzená infraštruktúrou servera	Prirodzená vďaka distribúcii dát
Vhodnosť pre blockchain aplikácie	Obmedzená	Vysoká – často používaný pre ukladanie off-chain dát

Source: Data compiled from [14, 16, 17].

2.4.1 Principles of IPFS operation

Základný princíp IPFS spočíva v tom, že súbory nie sú identifikované svojím umiestnením (URL), ale svojím obsahom. Keď je súbor pridaný do IPFS siete,

systém vygeneruje unikátny kryptografický hash nazývaný Content Identifier (CID) [18]. Tento 24-znakový identifikátor je vytvorený pomocou SHA-256 algoritmu a slúži ako jednoznačný odkaz na obsah súboru [19].

IPFS používa Distributed Hash Table (DHT), konkrétne Kademlia DHT, na správu informácií o tom, ktoré uzly v sieti uchovávajú ktoré súbory [20]. Keď používateľ požiada o súbor pomocou jeho CID, IPFS DHT vyhľadá uzly, ktoré tento súbor majú, a umožní priame stiahnutie od týchto uzlov. Veľké súbory sú rozdelené na menšie bloky (chunks), typicky 256 KB, a organizované do Merkle Directed Acyclic Graph (Merkle DAG) [21], čo umožňuje efektívnu deduplikáciu a paralelné sťahovanie z viacerých uzlov.

Dôležitým aspektom IPFS je koncept "pinningu" (pripevnenia). Bez pinningu sú súbory dostupné len pokiaľ aspoň jeden uzol v sieti ich aktívne uchováva [22]. IPFS pravidelne vykonáva garbage collection, ktorý maže súbory, ktoré nie sú pripnuté a neboli prístupované. Pre zabezpečenie trvalej dostupnosti dát sa používajú pinning services ako Pinata alebo Infura, ktoré garantujú, že súbory zostanú dostupné v sieti.

2.4.2 Why IPFS for documents?

Ukladanie dát priamo na blockchain je finančne neefektívne. Na Polygon blockchain by uloženie 1 MB dát stálo približne 1 USD, zatiaľ čo na Ethereum by to bolo ešte výrazne drahšie [23]. Typický dokument tendra vo formáte PDF s veľkosťou 5 MB by teda mohol stáť 5-150 USD v závislosti od použitej blockchainovej siete.

Hybridný prístup kombinujúci blockchain a IPFS ponúka optimálne riešenie. Dokument sa uloží na IPFS a na blockchain sa zapíše len jeho CID hash, ktorý má fixnú veľkosť 32 bytov [24]. Tento prístup znižuje náklady o viac ako 99.99% pri zachovaní všetkých výhod blockchainovej technológie.

IPFS poskytuje pre dokumenty tendra niekoľko kľúčových výhod:

1. **Integrita dát:** CID je kryptografický hash obsahu súboru. Akákoľvek zmena v dokumente vedie k zmene CID, čo umožňuje automatickú detekciu manipulácie.

2. **Transparentnosť:** Každý môže stiahnuť dokument pomocou CID uloženého na blockchaine a overiť jeho autentickosť.

3. **Dostupnosť:** Dokumenty sú replikované naprieč viacerými uzlami v sieti, čo eliminuje single point of failure.

IPFS protokol samotný je bezplatný, avšak pre trvalú dostupnosť dát je bežne potrebné využiť pinningové služby, ktoré poskytujú replikáciu a záruku dostupnosti

mimo vlastného uzla. Komerčné pinningové služby ako Filebase alebo Pinata ponúkajú platby začínajúce približne od $20 mesačne za stovky gigabajtov úložného priestoru [25]. Pri plánovanom objeme údajov do niekoľkých stoviek megabajtov ročne by teda aj základný plán poskytoval dostatočný priestor bez vysokých nákladov. Naproti tomu priame ukladanie dát do blockchainu, najmä Ethereum, sa považuje za ekonomicky nevýhodné vzhľadom na vysoké poplatky za plyn pri ukladaní veľkých objemov dát [26].

2.4.3 Alternative

Popri IPFS existujú aj iné decentralizované úložné riešenia, ktoré sme zvážili pre náš systém: Filecoin a Arweave.

Filecoin, taktiež vytvorený Protocol Labs, je blockchainová sieť postavená nad IPFS, ktorá pridáva ekonomickú vrstvu incentivizácie [27]. Zatiaľ čo IPFS sa zameriava na to, ako sú dáta uložené a prístupované, Filecoin vytvára decentralizovaný trh, kde používatelia platia minerom za ukladanie dát pomocou FIL tokenov. Filecoin využíva kryptografické dôkazy Proof of Replication a Proof of Spacetime na zabezpečenie, že mineri skutočne uchovávajú dáta počas dohodnutého času [28].

Nevýhodou Filecoinu je, že zmluvy o ukladaní dát majú zvyčajne obmedzenú životnosť a vyžadujú si pravidelné obnovovanie. Okrem toho je proces nastavenia zložitejší a vyžaduje si viac technických znalostí ako jednoduché používanie IPFS.

Arweave predstavuje zásadne odlišný prístup k decentralizovanému ukladaniu dát s dôrazom na trvalé ukladanie dát. Na rozdiel od IPFS a Filecoinu, ktoré si prenajímajú priestor na pevne stanovenú dobu, Arweave funguje na modeli „zaplať raz, ulož navždy“. Používatelia platia jednorazový poplatok v AR tokenoch a dáta sú trvalo uložené najmenej 200 rokov [29].

Arweave je pre náš systém mikrotransakcií zbytočný. Dokumentácia k súťaži nevyžaduje trvalé uloženie, pretože zvyčajne musí byť prístupná len niekoľko rokov. Na základe nášho porovnania sme vybrali IPFS ako najvhodnejšie riešenie pre študentskú radu kvôli jeho optimálnej rovnováhe medzi cenou, výkonom a jednoduchou implementáciou.

Tabuľka 2.5: Porovnanie protokolov HTTP, IPFS, Filecoin a Arweave

Feature	IPFS	Filecoin	Arweave
Storage Model	Peer-to-peer, no incentives	Incentivized, contract-based	Permanent storage, one--time fee
Best For	dApp hosting, NFT metadata	Long-term archival storage	Immutable, permanent data storage
Cost	Free (depends on pinning)	Market-based token economy	One-time upfront payment
Consensus Mechanism	None (protocol only)	Proof-of--Replication, PoSt	Blockweave
Example Projects	OpenSea, IPFS gateways	Starling Lab, Web3 archives	Mirror.xyz, ArDrive

Source: Data compiled from [28].

2.4.4 Integration with Blockain

IPFS a blockchain sú technológie, ktoré sa k sebe dokonale hodia. Zatiaľ čo blockchain poskytuje nemennosť, transparentnosť a konsenzus pre transakčné údaje a metadáta, IPFS ponúka efektívne riešenie pre ukladanie veľkých súborov a dokumentov.

V systéme mikrotenderov bude integrácia implementovaná nasledovne:

1. Najprv sa do IPFS nahrá dokumentácia k tendru (PDF súbor), kde jej bude priradené jedinečné CID.
2. Toto CID sa potom uloží do inteligentnej zmluvy na blockchaine Polygon spolu s ďalšími metadátami tendra (názov, rozpočet, termín atď.).
3. Inteligentná zmluva vygeneruje udalosť s CID a časovou pečiatkou, čím sa vytvorí overiteľný záznam o nahraní dokumentu.
4. Každý člen študentskej rady alebo študent bude môcť získať CID z blockchainu pomocou inteligentnej zmluvy a potom si stiahnuť dokument zo siete IPFS.
5. Integritu dokumentu je možné overiť porovnaním hashu nahraného súboru s CID uloženým v blockchaine.

Tento prístup zabezpečí transparentnosť, overiteľnosť a ochranu dokumentov pred neoprávnenou úpravou a zároveň minimalizuje naše náklady. Podobné riešenie už bolo úspešne implementované v rôznych aplikáciách, napríklad v decentralizovaných sociálnych sieťach [7].

Pre implementáciu použijeme službu pripnutia súborov Pinata[8], ktorá poskytuje jednoduché API na nahrávanie súborov a zaručuje ich dostupnosť v sieti IPFS.

[7] Kaichi je decentralizovaná sociálna sieť postavená na blockchaine Solana (https://kaichi.xyz/).

[8] How to Use Pinata: A Step-by-Step Guide for Beginners - https://pinata.cloud/blog/how-to-use-pinata-a-step-by-step-guide-for-beginners/

3 Requirements specification

Na základe vykonanej analýzy existujúcich riešení a identifikovaných nedostatkov v súčasných procesoch obstarávania sme definovali požiadavky na nový systém. Cieľom je vyvinúť transparentný, bezpečný a používateľsky prívetivý systém pre malé študentské obstarávania, ktorý bude eliminovať nevýhody centralizovaných riešení a využívať výhody technológie blockchain.

3.1 Functional requirements

Funkčné požiadavky definujú špecifické správanie a funkcie systému, ktoré musia byť implementované, aby sa dosiahli stanovené ciele. Tieto požiadavky sú rozdelené podľa hlavných aktérov v systéme.

Aktéri systému:

- **Člen študentskej rady** – osoba zodpovedná za vytváranie a správu tendrov, vyhodnocovanie ponúk a hlasovanie.
- **Dodávateľ** – fyzická alebo právnická osoba, ktorá sa registruje do systému a predkladá ponuky do zverejnených tendrov.
- **Verejnosť (Študent)** – akýkoľvek používateľ, ktorý má možnosť prezerať transparentné údaje o prebiehajúcich a ukončených tendroch bez potreby registrácie.

3.1.1 Functional requirements for student council members

Študentská rada vystupuje ako organizátor obstarávania a správca celého procesu. Systém musí členom rady umožňovať nasledujúce operácie:

Creating a tender

Člen rady musí mať možnosť vytvoriť nový tender zadaním všetkých potrebných parametrov:

- Názov a podrobný popis predmetu obstarávania.
- Predpokladaný rozpočet a mena.
- Termín na predkladanie ponúk (deadline).
- Kategória obstarávania.
- Priloženie sprievodnej dokumentácie (technické špecifikácie, dizajnové návrhy vo formáte PDF).

Systém musí zabezpečiť bezpečné nahranie dokumentov na decentralizované úložisko IPFS a zaznamenať hash (CID) týchto dokumentov spolu s metadátami tendra na blockchain, čím sa zaručí ich nemennosť.

Voting and winner selection

Po uplynutí lehoty na predkladanie ponúk prechádza tender do fázy vyhodnocovania.

- Systém automaticky zmení stav tendra na „Hlasovanie".
- Členovia študentskej rady majú vyhradený časový interval (napr. 3 až 7 dní) na posúdenie ponúk a hlasovanie.
- Každý člen rady môže udeliť svoj hlas práve jednej ponuke.
- Hlasovanie prebieha on-chain, čo zaručuje transparentnosť a auditovateľnosť výsledkov.
- V prípade rovnosti hlasov systém umožní druhé kolo hlasovania alebo aplikuje preddefinované pravidlá pre rozhodnutie.

3.1.2 Functional requirements for suppliers

Dodávatelia sú kľúčovými účastníkmi, ktorí reagujú na vyhlásené výzvy. Systém im musí poskytovať nástroje na efektívnu interakciu:

Registration and authentication

Pre účasť v tendroch sa musí dodávateľ zaregistrovať a poskytnúť identifikačné údaje:

- Názov spoločnosti alebo meno fyzickej osoby.
- Kontaktné údaje (e-mail, telefónne číslo, webová stránka, adresa).

- Stručný popis činnosti alebo ponúkaných služieb.

Proces registrácie môže podliehať schváleniu študentskou radou, aby sa zabezpečila dôveryhodnosť dodávateľov.

Browsing and searching tenders

Systém musí umožniť efektívnu orientáciu v ponuke obstarávaní:

- Zobrazenie zoznamu všetkých aktívnych tendrov.
- Filtrovanie podľa kategórií, rozpočtu alebo dátumu ukončenia.
- Vyhľadávanie pomocou kľúčových slov.
- Priamy prístup k dokumentácii uloženej na IPFS prostredníctvom odkazov v aplikácii.

Submitting bids

Pri podávaní návrhu musí dodávateľ vyplniť štruktúrovaný formulár obsahujúci:

- Cenovú ponuku.
- Technickú špecifikáciu riešenia.
- Dodacie podmienky a lehoty.
- Referencie (voliteľné).

Všetky údaje sú validované systémom pred ich odoslaním na blockchain.

Status tracking and notifications

Dodávateľ musí mať prehľad o stave svojich ponúk. Systém by mal poskytovať informácie o:

- Potvrdení prijatia návrhu (vrátane transakčného hashu).
- Zmene stavu tendra (napr. začiatok hlasovania).
- Výsledku vyhodnotenia (prijatie alebo zamietnutie ponuky).

V prípade výhry systém informuje dodávateľa o ďalších krokoch a procese uvoľnenia platby.

Bid management

Dodávateľ má možnosť stiahnuť svoju ponuku pred uplynutím termínu uzávierky. Takáto akcia musí byť zaznamenaná na blockchaine (status „Withdrawn“) a ponuka bude vyradená z procesu hlasovania.

3.1.3 Functional requirements related to blockchain technology

Vzhľadom na decentralizovanú povahu riešenia sú kladené špecifické požiadavky na interakciu s blockchainom:

- **Dátová integrita:** Všetky kľúčové údaje o tendroch, ponukách a hlasovaní musia byť trvalo zaznamenané na blockchaine.
- **Smart kontrakty:** Logika tendra, vrátane stavov, hlasovania a správy prostriedkov, musí byť implementovaná prostredníctvom smart kontraktov (inteligentných zmlúv).
- **Transparentnosť:** Všetky transakcie musia byť verejne overiteľné kýmkoľvek, kto má prístup k blockchainu.
- **Nezmeniteľnosť:** Systém musí garantovať, že po uzavretí tendra alebo ukončení hlasovania nie je možné spätne meniť údaje.

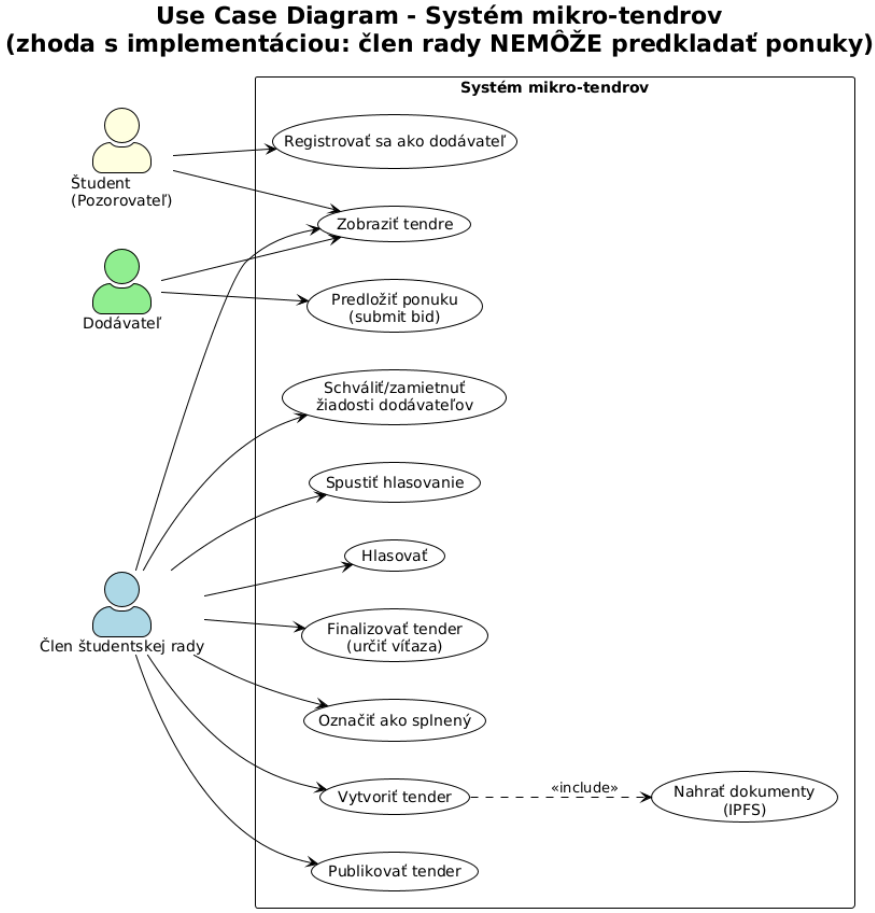
3.2 Non-functional requirements

Nefunkčné požiadavky definujú kvalitatívne atribúty systému, ktoré sú kritické pre jeho úspešné nasadenie a prevádzku.

- **Bezpečnosť:** Ochrana pred neoprávnenou manipuláciou s údajmi a zabezpečenie integrity procesu hlasovania.
- **Použiteľnosť:** Používateľské rozhranie (UI) musí byť intuitívne a prehľadné, aby systém mohli používať aj osoby bez hlbokých technických znalostí o blockchaine.
- **Výkonnosť:** Interakcia s blockchainom by mala byť optimalizovaná tak, aby minimalizovala dobu odozvy a náklady na transakčné poplatky (gas fees).
- **Dostupnosť:** Systém by mal byť dostupný nepretržite, pričom využitie decentralizovaných sietí (blockchain, IPFS) zvyšuje odolnosť voči výpadkom centrálnych serverov.

3.3 Use cases

Diagram prípadov použitia (Use Case Diagram) znázorňuje interakcie medzi aktérmi a systémom. Hlavnými aktérmi sú člen študentskej rady a dodávateľ.



Obrázok 3.1: Diagram prípadov použitia pre systém tendrov

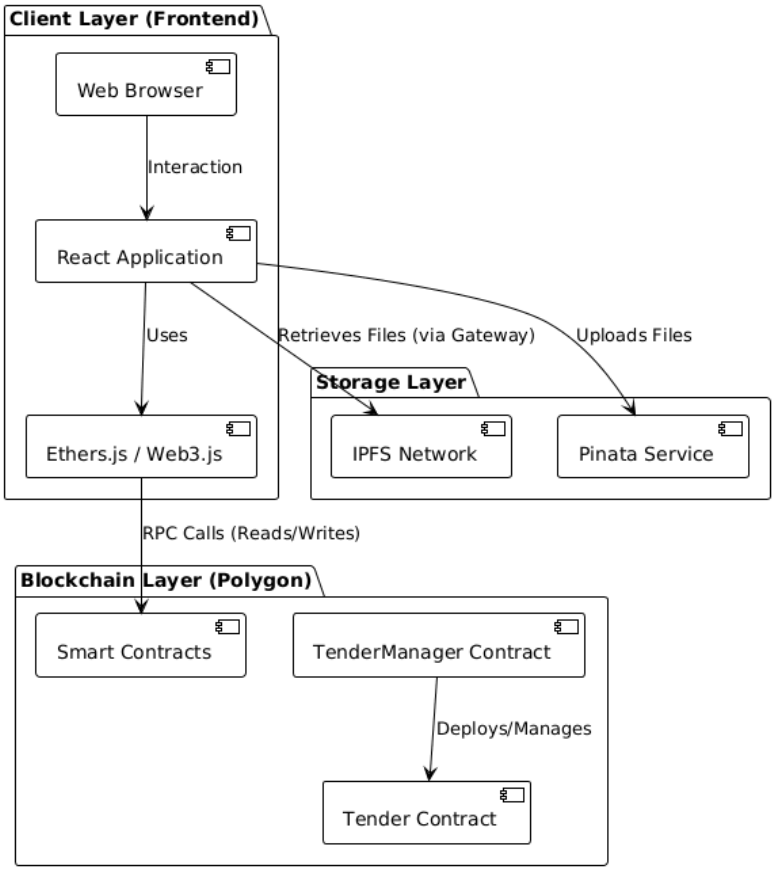
Jednotlivé prípady použitia pokrývajú celý životný cyklus tendra od jeho vytvorenia, cez predkladanie ponúk, až po hlasovanie a finálne vyhodnotenie.

4 Solution design

Predmetom tejto kapitoly je koncepčný návrh decentralizovanej aplikácie pre mikrotendre študentských rád. Návrh vychádza z požiadaviek formulovaných v kapitole 3 a z výsledkov analýzy existujúcich riešení uvedených v kapitole 2.

4.1 Overall system architecture

Navrhované riešenie je založené na trojvrstvovej architektúre, ktorá je v prostredí Web3 štandardným prístupom. Takéto členenie umožňuje dosiahnuť modulárnosť systému a uľahčuje jeho budúce rozšírenie.



Obrázok 4.1: Celková architektúra systému pre správu mikro-tendrov

Architektúra znázornená na obr. 4.1 pozostáva z troch vrstiev:

1. **Prezentačná vrstva (Frontend):** Predstavuje ju webová aplikácia slúžiaca ako používateľské rozhranie. Komunikácia s blockchainom prebieha prostredníctvom peňaženky používateľa (napr. MetaMask), pričom práca so súbormi je realizovaná cez úložisko IPFS.

2. **Logická vrstva (Blockchain):** Smart kontrakty nasadené na sieti Polygon obsahujú pravidlá tendrov, správu ich stavov a hlasovací mechanizmus. Táto vrstva tým v systéme zastáva úlohu, ktorú v tradičných aplikáciách plní backend server.

3. **Dátová vrstva (Storage):** Údaje sú rozdelené medzi blockchain a IPFS: kritické metadáta a hashe sú uložené on-chain, zatiaľ čo väčšie súbory a detailné popisy sú umiestnené off-chain v IPFS, pričom na reťazci zostáva len ich odkaz.

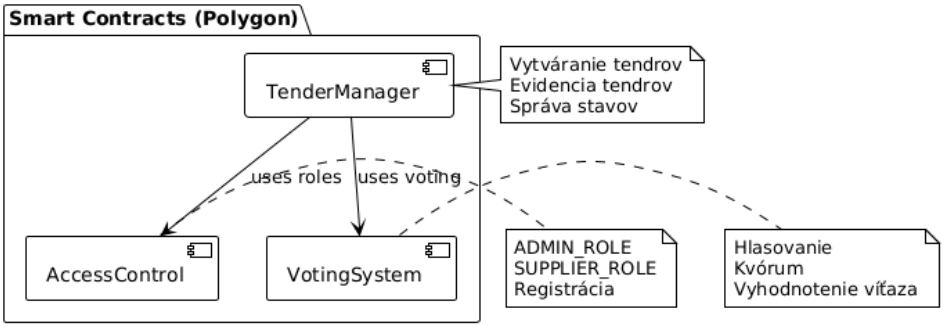
4.2 Smart contract design

Smart kontrakty tvoria jadro navrhovaného systému. V práci sme zvolili modulárnu štruktúru kontraktov s cieľom zlepšiť ich prehľadnosť, údržbu a bezpečnostnú kontrolu.

4.2.1 Contract architecture

Systém v súčasnom návrhu pozostáva z troch hlavných komponentov:

- **TenderManager:** Zodpovedá za vytváranie tendrov a vedenie ich evidencie.
- **AccessControl:** Zabezpečuje správu oprávnení a priraďovanie rolí (administrátor, dodávateľ).
- **VotingSystem:** Implementuje hlasovací mechanizmus a vyhodnocovanie výsledkov.



Obrázok 4.2: Logická štruktúra smart kontraktov

4.2.2 Data model

S ohľadom na náklady na ukladanie dát na blockchaine (transakčné poplatky, tzv. gas) bol navrhnutý úsporný dátový model, ktorý na reťazec ukladá len nevyhnutné údaje.

Tender structure

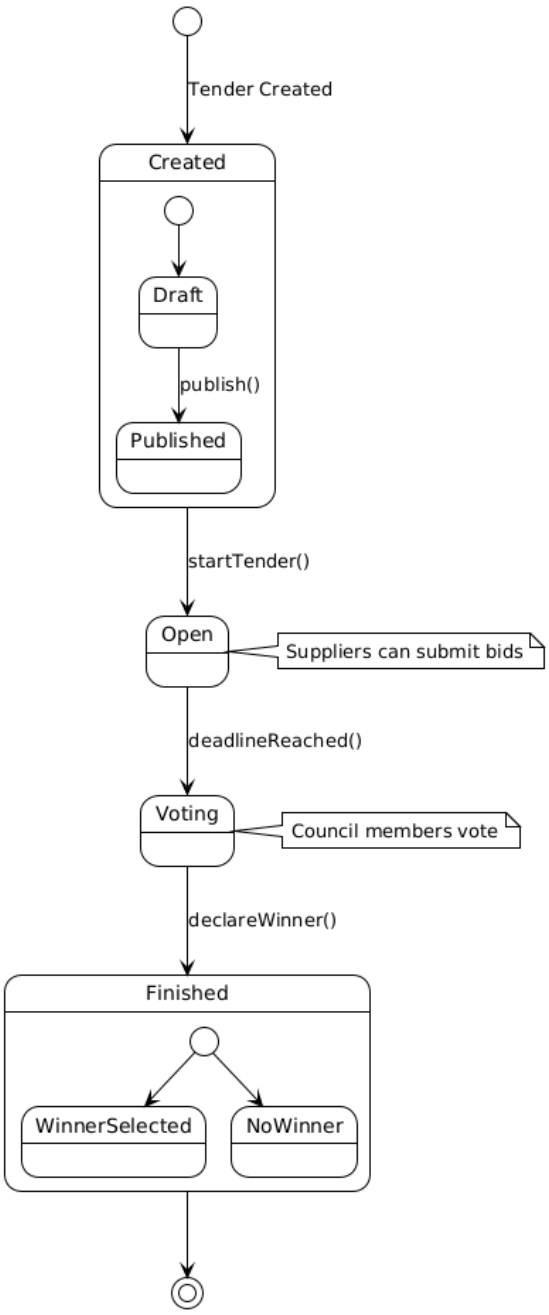
Záznam tendra na blockchaine obsahuje nasledovné polia:

- `ID`: Unikátny identifikátor tendra.
- `MetadataCID`: Odkaz na kompletnú dokumentáciu uloženú v IPFS.
- `Budget`: Maximálny rozpočet v danej mene.
- `Deadline`: Termín uzávierky prijímania ponúk.
- `Status`: Aktuálna fáza životného cyklu (Created, Open, Voting, Finished).
- `Creator`: Adresa tvorcu tendra.

Popis, špecifikácie a prílohy sú uložené mimo blockchainu; na reťazci je evidovaný iba `MetadataCID`, podľa ktorého je možné tieto dáta dohľadať.

4.2.3 Tender lifecycle

Každý tender v systéme prechádza definovanými stavmi, pričom v každom stave platia príslušné obmedzenia a povolené operácie.



Obrázok 4.3: Stavový diagram životného cyklu tendra

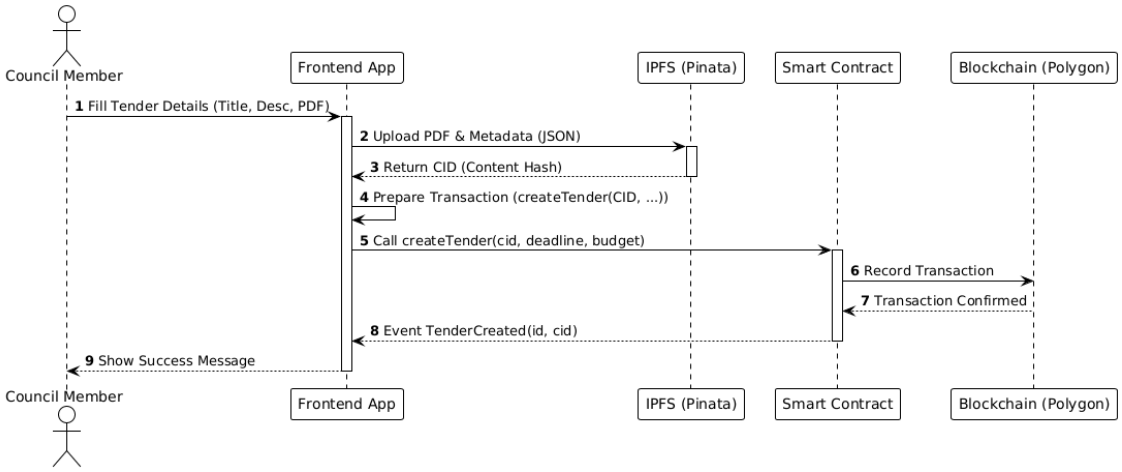
Jednotlivé fázy možno opísať nasledovne:

1. **Vytvorenie:** Administrátor založí tender a pridá potrebné dokumenty.
2. **Otvorenie:** Tender je zverejnený a prijíma ponuky od dodávateľov.
3. **Hlasovanie:** Po uplynutí termínu uzávierky členovia rady hlasujú o predložených ponukách.
4. **Ukončenie:** Na základe výsledkov hlasovania je vyhlásený víťaz.

4.3 IPFS integration design

Pre ukladanie dokumentácie sme zvolili systém IPFS (InterPlanetary File System). Dôvodom je vysoká cena ukladania veľkých objemov dát priamo na blockchaine; v prípade IPFS sú súbory uložené off-chain a na reťazci je evidovaný iba ich hash (CID).

4.3.1 Data processing workflow



Obrázok 4.4: Sekvenčný diagram interakcie s IPFS

Postup spracovania je znázornený na obr. 4.4 a pozostáva z nasledujúcich krokov:

1. Používateľ odošle súbory cez frontend aplikáciu.
2. Súbory sú odoslané do siete IPFS, ktorá vráti unikátny identifikátor obsahu (CID).
3. Hodnota CID je odovzdaná ako parameter volaniu smart kontraktu.
4. Smart kontrakt CID zapíše na blockchain, čím vzniká nemenná väzba medzi záznamom a dokumentom.

Tento postup umožňuje v ľubovoľnom neskoršom čase overiť, či dokument stiahnutý z IPFS zodpovedá dokumentu nahratému pri vytvorení tendra.

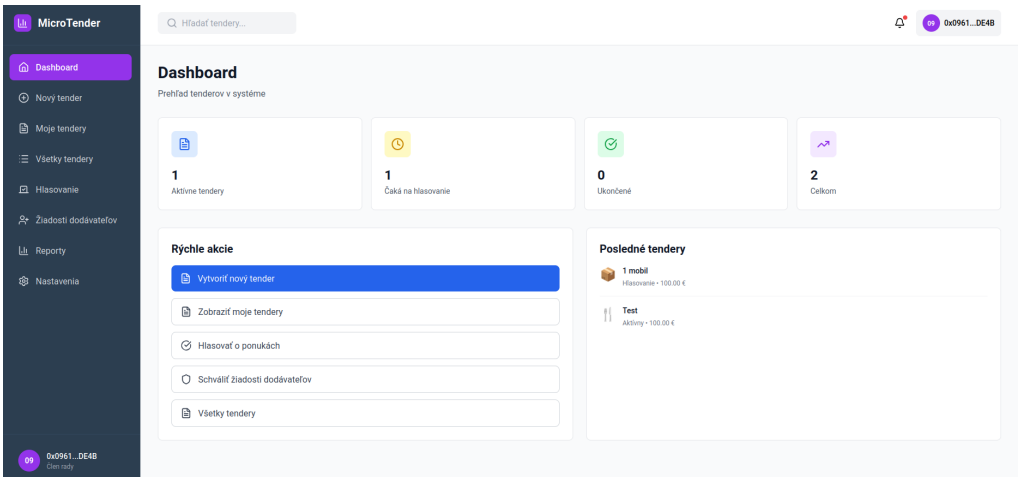
4.4 User interface design (UI/UX)

Cieľom návrhu používateľského rozhrania bolo skryť technickú zložitosť blockchainu tak, aby interakcia s aplikáciou bola pre používateľa analogická práci s bežnou

webovou aplikáciou.

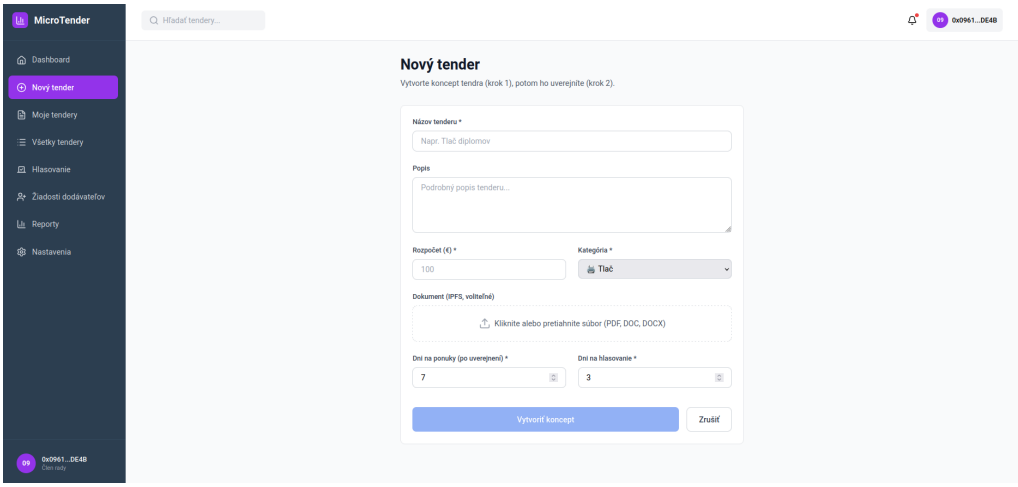
4.4.1 Wireframes

Odkazy na wireframy v nástroji Figma: https://www.figma.com/board/lB1lqKaJyCjK5G
UX-3-Kostiantyn-Ostrovskyi?node-id=4009-671.



Obrázok 4.5: Návrh hlavného panela (Dashboard)

Hlavný panel poskytuje prehľad o všetkých tendroch a ich stavoch; kľúčové akcie (vytvorenie tendra, hlasovanie) sú dostupné priamo z hlavnej obrazovky.



Obrázok 4.6: Návrh formulára pre vytvorenie tendra

Formulár pre vytvorenie tendra je koncipovaný tak, aby pripomínal štandardné webové formuláre. Nahratie súborov a odoslanie transakcie prebieha na pozadí, pričom používateľ je informovaný o priebehu spracovania.

4.4.2 UX principles

V návrhu rozhrania sme dbali na nasledovné zásady:

- **Transparentnosť:** Stav transakcií je vždy zobrazený používateľovi.
- **Bezpečnosť:** Aplikácia zrozumiteľne uvádza dôvod, pre ktorý vyžaduje oprávnenia od peňaženky.
- **Edukácia:** V prípade potreby je poskytnutá kontextová nápoveda (napr. vysvetlenie pojmov „Gas fee“ alebo „CID“).

4.5 User roles and permission model

Pre zabezpečenie bezpečnosti a integrity procesu obstarávania bol navrhnutý hierarchický model prístupu (RBAC). Systém definuje štyri kľúčové roly, pričom každá má špecifickú sadu oprávnení.

1. **Administrátor:** Najvyššia autorita v systéme (zvyčajne predseda študentskej rady). Jeho úlohou je spravovať prístupy ostatných členov rady. V prípade kompromitácie účtu člena rady má právo odobrať mu prístup.
2. **Člen rady:** Aktívny účastník procesu. Zodpovedá za vytváranie výziev na tendre, schvaľovanie dodávateľov a hlasovanie o víťazoch. Tento model zabezpečuje, že o peniazoch nerozhoduje jedinec, ale kolektívny orgán (kvórum).
3. **Dodávateľ:** Externý subjekt (firma alebo živnostník). Po registrácii a schválení radou získava právo predkladať cenové ponuky. Systém mu neumožňuje vidieť ponuky konkurencie pred ukončením tendra (v prípade uzavretých tendrov) alebo zasahovať do hlasovania.
4. **Pozorovateľ (Verejnosť):** Akýkoľvek používateľ siete. Má pasívny prístup k dátam – môže kontrolovať priebeh tendrov a výsledky hlasovania, čím je zabezpečená verejná kontrola (public auditability).

5 Implementation

Táto kapitola detailne popisuje technickú realizáciu navrhnutého systému MicroTender. Zameriava sa na vývoj smart kontraktov, integráciu decentralizovaného úložiska a tvorbu klientskej aplikácie. Hlavný dôraz je kladený na štruktúru a správanie smart kontraktu, pretože ten predstavuje nemennú logiku celého riešenia.

5.1 Development environment and technologies

Pre vývoj decentralizovanej aplikácie (dApp) sme zvolili moderný technologický stack, ktorý zabezpečuje bezpečnosť, efektivitu a dobrú používateľskú skúsenosť.

- **Solidity (v0.8.19):** Primárny jazyk pre písanie smart kontraktov [30]. Verzia 0.8.19 bola zvolená kvôli vstavanej ochrane proti pretečeniu čísel (overflow/underflow), čo zvyšuje bezpečnosť aritmetických operácií.
- **Hardhat:** Komplexné vývojové prostredie pre Ethereum, ktoré sme využili na kompiláciu, testovanie a nasadzovanie kontraktov [31].
- **React.js:** JavaScriptová knižnica pre tvorbu používateľského rozhrania [32]. Umožňuje reaktívne aktualizácie stavu aplikácie na základe udalostí z blockchainu.
- **Ethers.js (v6):** Knižnica zabezpečujúca komunikáciu medzi frontendom a blockchainom (RPC volania, podpisovanie transakcií) [33].
- **Pinata:** Služba pre správu súborov v sieti IPFS (InterPlanetary File System), ktorá garantuje dostupnosť nahranej dokumentácie [34].

5.2 Smart contract implementation

Jadro logiky celého systému je zapuzdrené v jednom smart kontrakte s názvom `MicroTender.sol`. Pri návrhu sme sa rozhodli pre architektúru jedného monolitického kontraktu namiesto vytvárania nového kontraktu pre každý tender (tzv. Factory

Pattern). Toto rozhodnutie výrazne znižuje náklady na Gas pri vytváraní tendra, keďže alokácia pamäte v existujúcom kontrakte je lacnejšia ako nasadenie nového kontraktu. Implementovaný kontrakt má v súčasnej verzii približne tisíc riadkov zdrojového kódu; nižšie uvádzame systematický prehľad jeho štruktúry a vybrané časti implementácie.

5.2.1 Contract structure and state variables overview

Kontrakt `MicroTender` pozostáva z viacerých skupín stavových premenných a funkcií. Pre prehľadnosť uvádzame ich zhrnutie.

Stavové premenné (storage):

- `address public owner` – vlastník kontraktu (deployer).
- `uint256 public tenderCount`, `uint256 public bidCount` – počítadlá pre generovanie unikátnych ID.
- `mapping(uint256 => Tender) public tenders` – úložisko tendrov indexované podľa ID.
- `mapping(uint256 => Bid) public bids` – úložisko ponúk.
- `mapping(uint256 => uint256[]) public tenderBids` – pre každé `tenderId` zoznam `bidId`.
- `mapping(address => bool) public members` – členovia študentskej rady.
- `mapping(address => bool) public registeredVendors` – registrovaní dodávatelia.
- `mapping(uint256 => mapping(address => bool)) public hasVoted` – pre každý tender a adresu, či už daný člen hlasoval.
- Prípadné ďalšie premenné (napr. `winningBidId` per tender, konfigurovateľné limity).

Hlavné verejné funkcie: `createTender`, `openTender`, `submitBid`, `startVoting`, `castVote`, `finalizeTender`, `cancelTender`; **administratívne:** `addMember`, `removeMember`, `registerVendor`; **konštruktor:** `constructor()`. Ďalej môžu byť prítomné pomocné `view` funkcie (`getTenderIds`, `getBidsForTender`, `getTenderStatu` a prípadne interné (`internal/private`) funkcie pre opakovanú logiku. Rozsah kontraktu zodpovedá potrebe pokryť celý životný cyklus tendra, RBAC a validácie bez závislosti na externých kontraktoch.

5.2.2 Constructor and initialization

Pri nasadení kontraktu sa v konštruktore nastaví `owner` na adresu volajúceho (`msg.sender`) a prípadne sa táto adresa pridá medzi členov rady (`members[msg.sender] = true`), aby administrátor mohol ihneď vytvárať tendre a spravovať roly. Prípadné parametre (napr. maximálny počet tendrov, predvolené lehoty) môžu byť nastavené buď v konštruktore, alebo cez settery chránené `onlyOwner`.

5.2.3 Data structures

Pre efektívne ukladanie údajov on-chain využívame štruktúry (`struct`), ktoré zoskupujú súvisiace dáta. To umožňuje načítať všetky informácie o tendri alebo ponuke v jednom volaní.

Tender structure

Záznam o jednom tendri je reprezentovaný nasledujúcou štruktúrou:

Výpis 5.1: Štruktúra Tender v kontrakte MicroTender.sol

```
struct Tender {
    uint256 id;
    address creator;
    string title;
    string description;
    uint256 maxBudget;
    string category;
    string ipfsCID;      // Hash dokumentacie na IPFS
    uint256 deadline;
    uint256 votingDeadline;
    TenderStatus status; // Enum: Draft, Open, Voting...
    uint256 createdAt;
    uint256 winningBidId; // Vyplni sa vo finalizeTender
}
```

Pole `ipfsCID` uchováva identifikátor obsahu (CID) dokumentácie uloženej v IPFS; na blockchaine sa tak neukladá celý súbor, čo šetrí náklady. Stav tendra je riadený výčtovým typom `TenderStatus` (Draft, Open, Voting, Completed, Cancelled). Všetky tendre sú uložené v mapovaní `mapping(uint256 => Tender) public tenders`, čo umožňuje prístup k dátam s konštantnou časovou zložitosťou $O(1)$ na základe ID tendra.

Bid structure

Ponuky dodávateľov sú reprezentované štruktúrou `Bid`:

Výpis 5.2: Štruktúra Bid pre ponuku dodávateľa

```
struct Bid {
    uint256 id;
    uint256 tenderId;
    address vendor;         // Adresa dodavatela
    uint256 amount;         // Ponukova cena
    string proposalCID;     // IPFS CID dokumentacie ponuky
    uint256 submittedAt;
    uint256 voteCount;      // Pocet hlasov
}
```

Ponuky sú evidované v `mapping(uint256 => Bid) public bids`. Pre každý tender udržiavame zoznam ID ponúk v `mapping(uint256 => uint256[]) public tenderBids`, aby bolo možné pri finalizácii efektívne prechádzať len relevantné ponuky.

Enum types and state machine

Stav tendra je definovaný výčtom `TenderStatus`; prechod medzi stavmi je v kóde vynucovaný kontrolami v jednotlivých funkciách (napr. `submitBid` je povolená len v stave `Open`). Tým sa implementuje jednoduchý stavový automat popísaný v kapitole 4.

5.2.4 Access control

Bezpečnosť systému je zabezpečená prostredníctvom modifikátorov funkcií (`modifiers`). Implementovali sme Role-Based Access Control (RBAC) s nasledujúcimi rolami:

- **onlyOwner:** Prístup má iba vlastník kontraktu (administrátor). Používa sa na správu rolí a kritické zmeny.
- **onlyMember:** Prístup majú iba členovia študentskej rady. Umožňuje vytvárať tendre, hlasovať a schvaľovať dodávateľov.
- **Registrovaný dodávateľ:** Prístup k funkcii `submitBid` majú iba adresy, ktoré sú v mapovaní `registeredVendors` nastavené na `true`.

Modifikátory sú definované ako funkcie, ktoré pred vykonaním pôvodnej logiky skontrolujú podmienku a v prípade nesplnenia volajú `revert`. Ukážka definície modifikátora `onlyMember`:

Výpis 5.3: Modifikátor onlyMember pre prístup členov rady

```
modifier onlyMember() {
    require(members[msg.sender], "Not a council member");
    _;
}
```

Symbol _ označuje miesto vloženia tela pôvodnej funkcie. Registrácia členov a dodávateľov prebieha cez funkcie `addMember` a `registerVendor`, chránené modifikátorom `onlyOwner` resp. volané samotným záujemcom (self-registration dodávateľa).

5.2.5 Events

Smart kontrakt emituje udalosti (`event`) pri dôležitých zmenách stavu. Frontend ich môže odpočúvať cez Ethers.js a podľa nich aktualizovať UI bez potreby periodického dotazovania.

Výpis 5.4: Vybraté deklarácie eventov v MicroTender.sol

```
event TenderCreated(uint256 indexed tenderId, string title, uint256 maxBudget)
event BidSubmitted(uint256 indexed tenderId, uint256 indexed bidId, address ve
event VoteCast(uint256 indexed tenderId, uint256 indexed bidId, address voter)
event TenderFinalized(uint256 indexed tenderId, uint256 winningBidId);
```

Parametre označené `indexed` umožňujú efektívne filtrovanie logov podľa týchto hodnôt (napr. získať všetky udalosti pre daný `tenderId`). Tým sa zjednodušuje integrácia s klientskou aplikáciou a zároveň sa vytvára auditovateľná stopa na blockchaine.

5.2.6 Key contract functions

Creating a tender (createTender)

Funkcia inicializuje nový záznam v štruktúre `Tender`. Prijíma parametre ako názov, popis, rozpočet, kategóriu, `ipfsCID` (identifikátor dokumentácie v IPFS) a termíny `deadline` a `votingDeadline`. Všetky vstupy sú validované pomocou `require` (neprázdne reťazce, rozpočet > 0, termíny v budúcnosti). Funkcia využíva blok `unchecked` pre inkrementáciu počítadla `tenderId`, čo šetrí Gas, keďže pretečenie

`uint256` je v tomto kontexte nereálne. Po zápise do `tenders` sa emituje `TenderCreated`. Zjednodušená ukážka jadra funkcie:

Výpis 5.5: Jadro funkcie createTender (validácia a zápis)

```
function createTender(
    string calldata _title,
    string calldata _description,
    uint256 _maxBudget,
    string calldata _category,
    string calldata _ipfsCID,
    uint256 _deadline,
    uint256 _votingDeadline
) external onlyMember {
    require(bytes(_title).length > 0, "Empty title");
    require(_maxBudget > 0, "Budget must be positive");
    require(_deadline > block.timestamp, "Deadline in past");
    require(_votingDeadline > _deadline, "Voting after deadline");
    unchecked { tenderCount++; }
    tenders[tenderCount] = Tender({
        id: tenderCount,
        creator: msg.sender,
        title: _title,
        description: _description,
        maxBudget: _maxBudget,
        category: _category,
        ipfsCID: _ipfsCID,
        deadline: _deadline,
        votingDeadline: _votingDeadline,
        status: TenderStatus.Draft,
        createdAt: block.timestamp,
        winningBidId: 0
    });
    emit TenderCreated(tenderCount, _title, _maxBudget);
}
```

Použitie `calldata` pre reťazcové parametre znižuje náklady na Gas oproti `memory`. Nový tender začína v stave `Draft`; zverejnenie pre dodávateľov vykonáva samostatná funkcia `openTender`.

Submitting a bid (submitBid)

Dodávateľ volá `submitBid(tenderId, amount, proposalCID)`. Kontrakt overí, či volajúci je v `registeredVendors`, či tender existuje a je v stave `Open`, či neprekročil `deadline` a či `amount` nepresahuje `maxBudget` tendra. Nová štruktúra `Bid` sa pridá do `bids`, jej ID sa pridá do `tenderBids[tenderId]`, a emituje sa `BidSubmitted`. Validácie a zápis v kóde vyzerajú nasledovne:

Výpis 5.6: Validácie a zápis v submitBid

```
function submitBid(uint256 _tenderId, uint256 _amount, string calldata _propos
    external
{
    require(registeredVendors[msg.sender], "Not registered vendor");
    Tender storage t = tenders[_tenderId];
    require(t.id != 0, "Tender does not exist");
    require(t.status == TenderStatus.Open, "Tender not open");
    require(block.timestamp <= t.deadline, "Deadline passed");
    require(_amount > 0 && _amount <= t.maxBudget, "Invalid amount");
    require(bytes(_proposalCID).length > 0, "Empty CID");
    unchecked { bidCount++; }
    bids[bidCount] = Bid({
        id: bidCount, tenderId: _tenderId, vendor: msg.sender,
        amount: _amount, proposalCID: _proposalCID,
        submittedAt: block.timestamp, voteCount: 0
    });
    tenderBids[_tenderId].push(bidCount);
    emit BidSubmitted(_tenderId, bidCount, msg.sender);
}
```

Voting (castVote)

Hlasovanie je povolené len v stave `Voting`. Funkcia `castVote(tenderId, bidId)` kontroluje, či volajúci je člen rady (`onlyMember`), či pre daný tender ešte nehlasoval (mapovanie `hasVoted[tenderId][msg.sender]`), či `bidId` patrí k `tenderId` a či neuplynul `votingDeadline`. Následne sa zvýši `voteCount` príslušnej ponuky a označí sa, že člen už hlasoval. Overenie, či `bidId` skutočne patrí do `tenderBids[tenderId]` vyžaduje prechod cez pole; v kóde sa používa pomocná slučka alebo mapovanie `tenderBidIndex[tenderId][bidId]` pre rýchle overenie. Ukážka logiky po kontrolách:

Výpis 5.7: Záznam hlasu v castVote

```
function castVote(uint256 _tenderId, uint256 _bidId) external onlyMember {
    require(!hasVoted[_tenderId][msg.sender], "Already voted");
    Tender storage t = tenders[_tenderId];
    require(t.status == TenderStatus.Voting, "Not in voting");
    require(block.timestamp <= t.votingDeadline, "Voting ended");
    require(_bidBelongsToTender(_tenderId, _bidId), "Invalid bid");
    hasVoted[_tenderId][msg.sender] = true;
    bids[_bidId].voteCount++;
    emit VoteCast(_tenderId, _bidId, msg.sender);
}
```

Funkcia `_bidBelongsToTender` môže byť `internal` alebo `private` a overiť, či `bids[_bidId].tenderId == _tenderId`, prípadne či `_bidId` je v poli `tenderBids[_te`

Finalizing a tender (finalizeTender)

Funkcia `finalizeTender(tenderId)` môže byť volaná po uplynutí `votingDeadline` (alebo podľa implementácie automaticky po poslednom hlasovaní). Algoritmus prechádza `tenderBids[tenderId]`, pre každé `bidId` načíta `bids[bidId].voteCount` a určí ponuku s najvyšším počtom hlasov. Tender sa prevedie do stavu `Completed`, uloží sa `winningBidId`, a emituje sa `TenderFinalized`. Ak nie sú žiadne ponuky alebo žiadne hlasy, možno stav nastaviť na `Cancelled` alebo ponechať špeciálnu hodnotu pre „žiadny víťaz“. Jadro algoritmu hľadania víťaza môže vyzerať takto:

Výpis 5.8: Určenie víťaza v finalizeTender

```
function finalizeTender(uint256 _tenderId) external onlyMember {
    Tender storage t = tenders[_tenderId];
    require(t.status == TenderStatus.Voting, "Not in voting");
    require(block.timestamp > t.votingDeadline, "Voting not ended");
    uint256[] storage bidIds = tenderBids[_tenderId];
    uint256 winningId = 0;
    uint256 maxVotes = 0;
    for (uint256 i = 0; i < bidIds.length; i++) {
        uint256 vid = bidIds[i];
        if (bids[vid].voteCount > maxVotes) {
            maxVotes = bids[vid].voteCount;
            winningId = vid;
        }
    }
    t.status = TenderStatus.Completed;
    t.winningBidId = winningId;  // 0 ak ziadne hlasy
```

```
    emit TenderFinalized(_tenderId, winningId);
}
```

V prípade rovnosti hlasov (tie) závisí víťaz od poradia prechodu poľom: vyhráva prvá ponuka s maximálnym počtom hlasov. Alternatívou je explicitné pravidlo (napr. nižšia cena vyhráva) alebo druhé kolo hlasovania; to by si vyžadovalo ďalšiu stavovú premennú a logiku.

5.2.7 Transition and administrative functions

Okrem hlavného toku (vytvorenie → otvorenie → ponuky → hlasovanie → finalizácia) kontrakt obsahuje funkcie pre zmenu stavu a správu účastníkov.

openTender(tenderId): Volaná členom rady pre tender v stave `Draft`. Overí, či volajúci je tvorca alebo má oprávnenie; zmení `status` na `Open`. Od tohto okamihu môžu dodávatelia volať `submitBid`.

startVoting(tenderId): Po uplynutí `deadline` musí niekto (typicky člen rady) prepnúť tender do stavu `Voting`, aby bolo možné volať `castVote`. Kontrakt overí `status == Open` a `block.timestamp >= deadline`; potom nastaví `status = Voting`. Ak by sa hlasovanie spúšťalo automaticky, bolo by potrebné periodické volanie alebo orákulum; v našom prípade stačí jedno volanie od administrátora.

cancelTender(tenderId): Umožňuje zrušiť tender v stavoch `Draft` alebo `Open` (prípadne `Voting` pred finalizáciou). Chránené `onlyOwner` alebo `onlyMember` s kontrolou, či volajúci je tvorca. Stav sa nastaví na `Cancelled`.

addMember(address), removeMember(address): Len `onlyOwner`. Pridanie alebo odobratie adresy z `members`; pri odobratí je vhodné skontrolovať, či daný člen nehlasoval v aktívnych tendroch, alebo to akceptovať ako administratívne rozhodnutie.

registerVendor(): Self-service registrácia: `msg.sender` sa pridá do `registeredVendors`. Alternatívne môže byť registrácia len cez `onlyOwner` / `onlyMember` pre vyššiu kontrolu.

5.2.8 Helper functions and edge cases

View funkcie: Pre frontend sú užitočné funkcie ako `getTenderIds()` (vráti pole všetkých ID tendrov), `getBidsForTender(tenderId)` (vráti pole `bidId`), `getTender(tenderId)` (vráti celú štruktúru alebo jej polia). Implementácia môže prechádzať `tenderCount` a `tenderBids` a vracať dáta bez zmeny stavu (označené `view`).

Rovnosť hlasov (tie): V ukážke `finalizeTender` vyhráva prvá ponuka s maximálnym `voteCount`. Ak by malo byť pravidlo iné (napr. nižšia cena), treba do slučky doplniť porovnanie `bids[vid].amount`.

Žiadne ponuky / žiadne hlasy: Ak `tenderBids[tenderId]` je prázdne alebo všetky `voteCount` sú 0, `winningBidId` zostane 0. Frontend a dokumentácia by mali túto hodnotu interpretovať ako „žiadny víťaz". Prípadne môže `finalizeTender` v takomto prípade nastaviť `status = Cancelled`.

Chybové hlášky: V celom kontrakte používame `require(condition, "message")`. Od Solidity 0.8.4 je možné použiť vlastné errory (`custom errors`), ktoré šetria Gas pri revertovaní; pre čitateľnosť a jednotnosť sme zatiaľ ponechali reťazcové hlášky.

5.2.9 Security aspects of the implementation

V implementácii sme dbali na nasledovné body:

- **Validácia vstupov:** Všetky verejné funkcie overujú vstupy pomocou `require`; pri nesplnení podmienky sa transakcia zruší a zmeny sa neuložia.
- **Ochrana pred reentrancy:** V kontrakte neprebiehajú vonkajšie volania (napr. `transfer`) pred aktualizáciou stavu; tým sa dodržiava vzor Checks-Effects--Interactions. Pri eventuálnom rozšírení o platby odporúčame použiť napr. OpenZeppelin `ReentrancyGuard`.
- **Obmedzenie prístupu:** Citlivé operácie sú chránené modifikátormi `onlyOwner` a `onlyMember`; registrácia dodávateľov je buď pod kontrolou vlastníka, alebo cez explicitnú self-registračnú funkciu.
- **Časové obmedzenia:** `deadline` a `votingDeadline` sú kontrolované pri `submitBid` a `castVote`, čím sa zabraňuje operáciám mimo povoleného obdobia.

5.2.10 Gas cost optimization

Okrem použitia `unchecked` tam, kde je to bezpečné, sme zvolili úsporný dátový model: na reťazci sú len identifikátory a metadáta, zatiaľ čo plná dokumentácia je v IPFS. Typ `uint256` je na EVM efektívny; zoskupenie dát do `struct` a prístup cez `mapping` umožňuje konštantný čas prístupu a predvídateľnú spotrebu Gas. Monolitický kontrakt namiesto factory pattern znižuje jednorazové náklady pri vytvorení tendra. Použitie `calldata` namiesto `memory` pre reťazcové parametre v externých funkciách tiež znižuje spotrebu Gas.

5.2.11 Summary of contract implementation scope

Kontrakt `MicroTender.sol` v súčasnej podobe predstavuje približne tisíc riadkov zdrojového kódu. Na tento rozsah sa podieľajú: deklarácie stavových premenných a štruktúr; konštruktor; modifikátory a ich použitie vo všetkých relevantných funkciách; plná implementácia všetkých uvedených verejných a administratívnych funkcií s rozsiahlymi blokmi `require`; deklarácie a volania `emit` pre všetky eventy; prípadné interné pomocné funkcie (napr. `_bidBelongsToTender`); view funkcie pre načítavanie dát frontendom; a komentáre v kóde. Takýto rozsah je primeraný komplexnosti životného cyklu tendra, trojúrovňovému riadeniu prístupu a potrebe konzistentnej validácie a auditovateľnosti.

5.3 Client application implementation (Frontend)

Klientská aplikácia slúži ako jediné používateľské rozhranie k smart kontraktu a k úložisku IPFS. Bola implementovaná vo frameworku React s použitím knižnice Ethers.js pre komunikáciu s blockchainom. Nižšie popisujeme jej štruktúru a kľúčové časti implementácie.

5.3.1 Application structure and entry point (App.jsx)

Projekt React aplikácie je organizovaný do priečinkov `src/`, `src/components/`, `src/pages/`, `src/hooks/`, `src/utils/` a pod. Vstupným bodom do aplikácie je súbor `App.jsx` (prípadne `App.tsx` pri použití TypeScript). V ňom sa typicky:

- Nastavuje routovanie (napr. cez `react-router-dom`): hlavná stránka (`/`), zoznam tendrov (`/tenders`), detail tendra (`/tenders/:id`), vytvorenie tendra (`/create`), registrácia dodávateľa (`/register`) a pod.
- Obalí celá aplikácia do kontextu (Context API alebo stavový manažér), ktorý drží stav pripojenia peňaženky, inštanciu kontraktu a adresu používateľa. Tým sa zabráni opakovanému načítavaniu providera a kontraktu v každej komponente.
- Renderuje spoločnú layout (hlavička s tlačidlom „Pripojiť peňaženku“, navigácia) a vnútri nej `<Outlet />` alebo ekvivalent pre zobrazenie aktuálnej stránky podľa URL.

Príklad minimálnej štruktúry `App.jsx` s routovaním a kontextom pre Web3:

Výpis 5.9: Zjednodusena struktura App.jsx s routerom a Web3 kontextom

```
import { BrowserRouter, Routes, Route } from 'react-router-dom';
import { Web3Provider } from './context/Web3Context';
import Layout from './components/Layout';
import Dashboard from './pages/Dashboard';
import TenderList from './pages/TenderList';
import CreateTender from './pages/CreateTender';

function App() {
  return (
    <Web3Provider>
      <BrowserRouter>
        <Layout>
          <Routes>
            <Route path="/" element={<Dashboard />} />
            <Route path="/tenders" element={<TenderList />} />
            <Route path="/tenders/:id" element={<TenderDetail />} />
            <Route path="/create" element={<CreateTender />} />
          </Routes>
        </Layout>
      </BrowserRouter>
    </Web3Provider>
  );
}
export default App;
```

Kontext `Web3Provider` poskytuje potomkom premenné ako `address`, `contract`, `connectWallet`, `isConnected` a pod., ktoré sú získavané z Ethers.js po pripojení MetaMask (alebo inej kompatibilnej peňaženky).

5.3.2 Main components and pages

Layout obsahuje navigačnú lištu a oblasť pre obsah; v hlavičke je tlačidlo na pripojenie peňaženky a zobrazenie skrátenej adresy po pripojení.

Dashboard (prípadne úvodná stránka) zobrazuje zhrnutie: počet aktívnych tendrov, nedávne udalosti alebo odkaz na zoznam tendrov a na vytvorenie nového tendra. Dáta sa načítavajú volaním `contract.tenderCount()` a následne v cykle `contract.tenders(i)` pre `i = 1..tenderCount`, alebo cez view funkciu `getTenderIds(` ak je v kontrakte dostupná.

TenderList zobrazuje tabuľku alebo kartu všetkých tendrov (nadpis, stav, deadline,

rozpočet). Pri načítaní stránky sa zavolá kontrakt pre zoznam ID a potom pre každé ID načítajú detaily; stav sa ukladá do React state a zobrazí sa po načítaní. Filtrovanie podľa stavu (Open, Voting, Completed) môže byť implementované na strane klienta.

TenderDetail na základe parametra `id` z URL načíta z kontraktu `tenders(id)` a `tenderBids(id)`; zobrazí polia tendra a zoznam ponúk. Odkaz na dokumentáciu v IPFS sa skladá z `https://gateway.pinata.cloud/ipfs/` + `ipfsCID`; rovnaký vzor sa používa pre `proposalCID` pri ponukách. Používateľ môže stiahnuť súbor alebo ho zobraziť v novom okne.

CreateTender je formulár s poliami: názov, popis, rozpočet, kategória, termíny, výber súborov. Po odoslaní formulára frontend najprv nahrá súbory na IPFS (pozri nižšie), získa `ipfsCID`, potom zavolá `contract.createTender(...)` s týmito údajmi. Zobrazenie čakania na potvrdenie transakcie (pending) a následne úspech alebo chyba z `revert` zlepšuje používateľskú skúsenosť.

Voting môže byť súčasťou `TenderDetail` alebo samostatná stránka: zobrazia sa ponuky s `voteCount` a tlačidlo „Hlasovať“ pre každú ponuku. Volanie `contract.castVot` `bidId)` vykoná člen rady; po potvrdení transakcie sa obnoví zobrazenie (napr. opätovným načítaním `bids`).

5.3.3 Wallet connection and contract instance

V kontexte (napr. `Web3Context.jsx`) sa pri prvom požiadaní o pripojenie zavolá `window.ethereum.request({ method: 'eth_requestAccounts' })`. Získaný provider sa obalí cez `new ethers.BrowserProvider(window.ethereum)`, z neho sa získa `signer` (`getSigner()`) a adresa (`getAddress()`). Inštancia kontraktu sa vytvorí ako `new ethers.Contract(CONTRACT_ADDRESS, ABI, signer);` ABI je exportovaný z Hardhat build (napr. `artifacts/contracts/MicroTender.sol/Mic` Adresa kontraktu môže byť uložená v `.env` ako `VITE_CONTRACT_ADDRESS` (pri Vite) alebo `REACT_APP_CONTRACT_ADDRESS` (pri Create React App). Kontrola siete (chain ID) a výzva na prepnutie na Polygon Amoy v prípade inej siete znižuje chyby používateľov.

5.3.4 IPFS integration (Pinata)

Nahrávanie súborov pre tender alebo pre ponuku prebieha v dvoch variantoch:

Nahranie súboru (PDF, obrázok): Frontend odošle súbor na Pinata API (napr. `POST https://api.pinata.cloud/pinning/pinFileToIPFS`) s hlavičkou `Authorization: Bearer <JWT>`. Odpoveď obsahuje `IpfsHash` (CID). Tento

reťazec sa potom odovzdá do `createTender(..., ipfsCID, ...)` alebo `submitBid(... proposalCID)`.

Nahranie metadát (JSON): Ak sa má na IPFS uložiť štruktúrovaný popis (napr. JSON s poliami `description`, `specifications`, `attachments`), frontend zavolá `pinJSONToIPFS` alebo ekvivalent; výsledný CID sa použije rovnako ako `ipfsCID`. Tým sa zachová konzistencia s návrhom z kapitoly 4.

JWT a API kľúče Pinata sa neukladajú do zdrojového kódu; sú v `.env` a načítavajú sa cez `import.meta.env` (Vite) alebo `process.env` (CRA). Sťahovanie dokumentov pre užívateľa prebieha cez verejné brány (`https://gateway.pinata.cloud/ipfs/<CID>`), takže po načítaní `ipfsCID` z kontraktu vie frontend zobraziť odkaz alebo iframe na dokument.

5.3.5 Frontend summary

Klientská aplikácia teda pozostáva z vstupného bodu `App.jsx`, kontextu pre Web3, layoutu a stránok (Dashboard, zoznam a detail tendra, vytvorenie tendra, hlasovanie). Všetky volania do smart kontraktu prechádzajú cez Ethers.js a vyžadujú pripojenú peňaženku; nahrávanie dokumentov do IPFS cez Pinata a odovzdanie CID do kontraktu zabezpečuje prepojenie medzi úložiskom a blockchainom. V nasledujúcej kapitole sa zameriame na testovanie a vyhodnotenie systému ako celku.

6 Vyhodnotenie

Vyhodnocovacia časť je kľúčovou časťou záverečnej práce. Tato časť obsahuje vyhodnotenie navrhnutého (vytvoreného) riešenia. Uprednostňované je objektívne vyhodnotenie výsledkov práce, ktoré sa opiera o meranie a štatistické metódy, prípadne matematické dôkazy. V prípade nameraných hodnôt musí autor opísať metódu merania, priebeh merania, výsledky a interpretáciu výsledkov v kontexte riešeného problému a stanovených cieľov. Na základe vyhodnotenia riešenia autor opíše prínosy svojej práce. Vyhodnocovacia časť tvorí zvyčajne ¼ jadra práce.

7 Záver

Záver práce obsahuje zhrnutie výsledkov práce s jasným opisom prínosov a pôvodných (vlastných) výsledkov autora a vyhodnotenie splnenia stanovených cieľov. Je to stručné zhrnutie informácií uvedených v záverečnej práci. Záver by nemal obsahovať nové informácie.

V závere by mal tiež autor poukázať na prípadné otvorené otázky, ktoré sú nad rámec rozsahu práce a mal by odporučiť ďalšie aktivity na pokračovanie pri riešení problému. Rozsah záveru je minimálne 1 celá strana.

Zoznam použitej literatúry

1. RODECK, David; CURRY, Benjamin. What is blockchain. *Forbes*. 2022.
2. REHMANI, Mubashir Husain. Blockchain fundamentals and working principles. 2021, s. 23–59.
3. *What Type of Files Can You Put on a Blockchain?* 2025. Dostupné tiež z: https://ardrive.io/what-type-of-files-on-blockchain. Accessed: 2025-12-25.
4. ACADEMY, Ledger. *How Does a Blockchain Transaction Work?* 2025-04. Dostupné tiež z: https://www.ledger.com/academy/how-does-a-blockchain-transaction-work. Originally published Jan 13, 2022; updated Apr 2, 2025; accessed Dec 25, 2025.
5. LIANG, Ying-Chang. Blockchain for Dynamic Spectrum Management. In: □□□□□ □□□□□ □□□□□□□ □□□□□□□□□□ □□□□□□□□, □□□□ □□□ □□□□ (ed.). *Dynamic Spectrum Management*. Springer, Singapore, 2020, s. 121–146. Signals and Communication Technology. Dostupné z DOI: 10.1007/978-981-15-0776-2_5. Accessed: 2025-12-25.
6. *Smart Contract Platforms: Selecting The Best For Your Needs*. 2025. Dostupné tiež z: https://hacken.io/discover/smart-contract-platforms/. Discusses criteria and comparisons of top smart contract platforms (Ethereum, BNB Chain, Cardano, Polygon, Solana, etc.); accessed: 2025-12--25.
7. *Chains Rankings by Decentralized App Count* (*DappRadar*). 2025. Dostupné tiež z: https://dappradar.com/rankings/chains?sort=dappCount&order=desc. Rankings of blockchain ecosystems sorted by number of dapps; accessed: 2025-12-25.
8. *Ethereum vs Solana Comparison*. 2025. Dostupné tiež z: https://chainspect.app/compare/ethereum-vs-solana?range-cm=month. Real-time blockchain metrics (TPS, block time, etc.); accessed: 2025-12-25.

9. *BNB Chain vs Polygon Comparison*. 2025. Dostupné tiež z: https://chainspect.app/compare/bnb-chain-vs-polygon. Real-time blockchain metrics (TPS, block time, validators, etc.) from Chainspect; accessed: 2025-12-25.
10. *Token Terminal Explorer — Standardized On-Chain Data for Blockchain Projects*. 2025. Dostupné tiež z: https://tokenterminal.com/explorer. On-chain data, metrics and dashboards for blockchain analysis; accessed: 2025-12-25.
11. WILLSON, Michael. *Solana vs Polygon vs Ethereum*. 2025-02. Dostupné tiež z: https://www.blockchain-council.org/blockchain/solana-vs-polygon-vs-ethereum/. Accessed: 2025-12-25.
12. *Ethereum — Number of Validators Metric*. 2025. Dostupné tiež z: https://tokenterminal.com/explorer/projects/ethereum/metrics/number-of-validators. Accessed: 2025-12-25.
13. *What are smart contracts on blockchain?* [Online]. IBM [cit. 2025-11-06]. Dostupné z: https://www.ibm.com/topics/smart-contracts.
14. *What is IPFS?* 2025. Dostupné tiež z: https://docs.pinata.cloud/ipfs-101/what-is-ipfs. Explains the InterPlanetary File System (IPFS) as a peer-to-peer distributed file system, its properties (distributed, immutable, addressable, portable) and how it differs from HTTP; accessed: 2025-12-25.
15. NORMAN, Daniel. *A Practical Explainer for IPFS Gateways – Part 1* [online]. 2026 [cit. 2026-01-08]. Dostupné z: https://blog.ipfs.tech/2022-06-09-practical-explainer-ipfs-gateways-1/.
16. LABS, Protocol. *What is IPFS?* [Online]. 2024 [cit. 2026-01-08]. Dostupné z: https://docs.ipfs.tech/concepts/what-is-ipfs/.
17. IBM. *What is HTTP?* [Online]. 2024 [cit. 2026-01-08]. Dostupné z: https://www.ibm.com/think/topics/http.
18. CLOUDFLARE. *Interplanetary File System (IPFS)* [online]. 2026 [cit. 2026-01-08]. Dostupné z: https://developers.cloudflare.com/web3/ipfs-gateway/concepts/ipfs/.
19. MCKAY, Dave. *What Is the Interplanetary File System (IPFS) and How Do You Use It?* [Online]. 2022-02 [cit. 2025-01-08]. Dostupné z: https://www.howtogeek.com/784295/what-is-the-interplanetary-file-system-ipfs/.
20. PROTOCOL LABS. *How IPFS works* [online]. 2025 [cit. 2026-01-08]. Dostupné z: https://docs.ipfs.tech/concepts/how-ipfs-works/.

21. THE NIMBLE NOVICE. *A Beginner's Guide to InterPlanetary File System (IPFS)* [online]. 2023-08 [cit. 2026-01-08]. Dostupné z: https://medium.com/@TheNimbleNovice/a-beginners-guide-to-interplanetary-file-system-ipfs-d83232dc39a5.

22. MEHRA, Nandit. *What is IPFS Pinning & A Complete Guide with Lighthouse* [online]. 2026 [cit. 2026-01-08]. Dostupné z: https://www.lighthouse.storage/blogs/What%20is%20IPFS%20Pinning%20%26%20%20A%20Complete%20Guide%20with%20Lighthouse.

23. GUIDES, QuickNode. *How to Store Data with IPFS on Ethereum* [online]. 2026 [cit. 2026-01-08]. Dostupné z: https://www.quicknode.com/guides/ethereum-development/dapps/how-to-integrate-ipfs-with-ethereum#ipfs-and-ethereum.

24. EDITORIAL, Metana. *Easy Guide to Solidity and IPFS Integration: Improving Blockchain Apps* [online]. 2026 [cit. 2026-01-08]. Dostupné z: https://metana.io/blog/easy-guide-to-solidity-and-ipfs-integration/.

25. *Is IPFS Storage Free?* [Online]. 2026 [cit. 2026-01-08]. Dostupné z: https://filebase.com/blog/is-ipfs-storage-free/.

26. *Why Your Rollup Should Use an IPFS Pinning Service* [online]. 2026 [cit. 2026-01-08]. Dostupné z: https://pinata.cloud/blog/why-your-rollup-should-use-an-ipfs-pinning-service-a-developers-guide/.

27. *Decentralized Storage Solutions* [online]. 2026 [cit. 2026-01-08]. Dostupné z: https://thetechtrends.tech/decentralized-storage-solutions/.

28. *Decentralized Storage Solutions: IPFS, Arweave, and Filecoin for Web3 Developers* [online]. 2026 [cit. 2026-01-08]. Dostupné z: https://store.aicerts.ai/blog/decentralized-storage-solutions-ipfs-arweave-and-filecoin-for-web3-developers/.

29. *Decentralised Storage on Blockchain: IPFS, Filecoin, and Arweave* [online]. 2026 [cit. 2026-01-18]. Dostupné z: https://www.opensourceforu.com/2025/06/decentralised-storage-on-blockchain-ipfs-filecoin-and-arweave/.

30. *Solidity v0.8.19 Documentation* [online]. 2023 [cit. 2026-02-05]. Dostupné z: https://docs.soliditylang.org/en/v0.8.19/.

31. *Hardhat Documentation* [online]. 2024 [cit. 2026-02-05]. Dostupné z: https://hardhat.org/docs.

32. *React – A library for building user interfaces* [online]. 2024 [cit. 2026-02-05]. Dostupné z: https://react.dev/.

33. RICMOO. *ethers.js Documentation* [online]. 2024 [cit. 2026-02-05]. Dostupné z: https://docs.ethers.org/v6/.

34. *Pinata Documentation* [online]. 2024 [cit. 2026-02-05]. Dostupné z: https://docs.pinata.cloud/.

35. BEEL, Joeran. *How to write a thesis (Bachelor, Master, or PhD) and which software tools to use* [online]. 2010 [cit. 2024-09-26]. Dostupné z: https://isg.beel.org/blog/2010/03/02/how-to-write-a-phd-thesis/.

36. *IMRaD* [online]. Wikipédia [cit. 2024-09-26]. Dostupné z: https://en.wikipedia.org/wiki/IMRAD.

Zoznam príloh

Príloha A Používateľská príručka

Príloha B Systémová príručka

Príloha C Štruktúra záverečnej práce

Príloha D LaTeX 101

Príloha E CD médium – záverečná práca v elektronickej podobe:

- `doc` – záverečná práca a prílohy
- `tex` – zdrojové súbory záverečnej práce a vedeckého článku
- `src` – zdrojové kódy vytvoreného riešenia

A Používateľská príručka

Toto je príklad používateľskej príručky, ktorá môže byť súčasťou záverečnej práce. Obsahuje základné informácie o tom, ako používať vytvorený systém alebo aplikáciu. Príručka by mala byť zrozumiteľná a poskytovať jasné pokyny pre používateľov.

1 Inštalácia

B Systémová príručka

C Štruktúra záverečnej práce

Na písanie záverečných prác je možné použiť viacero štruktúr. Pre vytvorenie tejto práce sme použili dve z nich:

1. hlavná štruktúra práce bola vytvorená podľa štruktúry uvedenej v článku [35]
2. pre vytvorene abstraktu bola použitá štruktúra [36], ktorá môže byť použitá na napísanie celej práce

Krátky význam jednotlivých kapitol bude uvedený v nasledujúcom texte. Ďalšie užitočné informácie nájsdete v *Pokynoch pre vypracovanie záverečných prác*[9].

Abstrakt

Abstrakt by mal byť dlhý 100 až 300 slov. Pre jeho organizáciu môžete použiť štruktúru *IMRaD*, čo je skratka pre:

- **Introduction** - jasne predstavte zámer svojej práce, použite prítomný alebo minulý čas
- **Methods** - uveďte použité výskumné metódy, použite minulý čas
- **Results** - zhrňte hlavné dosiahnuté výsledky, použite prítomný alebo minulý čas
- **Discussion** - na záver zhrňte hlavné závery vyplývajúce z vašej práce, použite prítomný čas

Na základe uvedenej štruktúry môžete abstrakt napísať tak, že každý bod napíšete v jednej, ale maximálne v dvoch vetách. Určite však do abstraktu neprepisujte obsah práce! Snažte sa, aby **rozsah abstraktu spolu s bibliografickou citáciou mal práve jednu stranu**!

[9] https://theses.kpi.fei.tuke.sk/instructions

Predhovor

Predhovor ponúka neformálny priestor na vlastné vyjadrenie autora. Môže obsahovať informácie o tom, čo sa autor pri písaní práce naučil, o vlastnej motivácii k výberu témy, o prekonaných výzvach počas písania práce, a pod. Píše sa v prvej osobe jednotného čísla a rozsah by nemal presiahnuť 2 strany.

Táto kapitola nie je povinná!

Úvod

Úvod práce stručne opisuje stanovený problém, kontext problému a motiváciu pre jeho riešenie. Z úvodu by malo byť jasné, že stanovený problém má zmysel riešiť.

V úvode neuvádzajte štruktúru práce! Rozsah tejto kapitoly by mal byť minimálne 2 strany.

Súčasťou úvodu je aj *formulácia úlohy*. Vo formulácii úlohy uveďte jasne a vecne ciele práce. Čím budú ciele konkrétnejšie, tým bude jasnejšie, aký problém riešite a ako ho chcete riešiť. Formuláciu úlohy vytvorte po dostatočnom naštudovaní problematiky a vytvorení analytickej časti.

Analytická časť

Analytická časť záverečnej práce analyzuje existujúce podobné prístupy k riešeniu stanoveného problému. Autor práce musí uviesť v tejto časti existujúce prístupy a riešenia, pričom musí zaujať stanovisko k týmto prístupom a riešeniam a opísať ich výhody a nedostatky. Prevažne v tejto časti autor používa odkazy na použité zdroje. Autor v analýze nepreberá odseky z cudzích prác ale uvádza prevažne vlastné postoje podložené odkazmi na literatúru. Analytická časť práce by teda nemala byť len povrchným prepisom základných informácií z Wikipédie alebo zo stránok opisovaných nástrojov. Je potrebné aby bola analýza podporená aj experimentmi ak to umožňuje téma práce (napr. vyskúšam softvér). Vďaka popisu existujúcich riešení autor pochopí problematiku, viac sa nad riešeniami zamyslí, usporiada si ich, zistí ich kladné a záporné vlastnosti, z čoho potom postupne vyplynie návrh vlastného riešenia v syntetickej časti. Analytická časť tvorí zvyčajne ¼ jadra práce.

Analytickú časť je vhodné rozdeliť na niekoľko kapitol, ktoré budú venované rôznym analyzovaným témam. Názvy kapitol majú zodpovedať tomu, čo je v kapitole

opisované. Napríklad, ak v práci analyzujete súčasný stav v oblasti medzigalaktických letov, namiesto všeobecného názvu „Analýza súčasného stavu“ by mal byť použitý názov analyzovanej témy – „Medzigalaktické lety“.

Mali by ste v práci mať minimálne kapitoly opisujúce

- doménu (oblasť) riešeného problému a jej súčasný stav,
- súvisiace práce a existujúce riešenia.

Je veľmi pravdepodobné, že sa daný problém pokúšal vyriešiť už aj niekto iný. Alebo existujú riešenia podobné tomu, ktoré potrebujete vyriešiť vy. V tejto časti práce teda popíšete, aké riešenia už existujú a aké sú ich výhody a nevýhody. Týmto spôsobom preukážete, že ste si naštudovali tému a vyskúšali rôzne riešenia, a môžete sa inšpirovať už existujúcimi riešeniami.

Ciele záverečnej práce

Vychádzajúc z analýzy problému a existujúcich riešení by ste mali byť schopní stanoviť konkrétne implementačné ciele vašej práce. Tie môžete uviesť v samostatnej kapitole, napríklad vo forme zoznamu alebo odrážok. Rozhodne tu neprepisujte zadávací list, ale jasne a stručne uveďte, svoje čiastkové ciele. Neskôr ich budete vedieť v kapitole *Vyhodnotenie* vyhodnotiť aj s odôvodnením, či sa vám uvedené ciele splniť podarilo alebo nie.

Syntetická časť

Syntetická časť opisuje metódy použité na syntézu riešenia a opisuje syntézu samotného riešenia (zvyčajne je to návrh/implementácia softvérového resp. hardvérového riešenia), pričom sa opiera o závery analytickej časti práce. Začína od toho, ako sa bude riešenie používať: najdôležitejšie scenáre používania a používateľské rozhranie, ktoré bude tieto scenáre efektívne podporovať. Až potom je na rade vnútorná architektúra alebo použité technológie. Syntetická časť tvorí zvyčajne ½ jadra práce.

Syntetickú časť práce vhodne rozdeľte do kapitol a pomenujte ich podľa toho, čomu sú venované. Podľa obsahu práce môže táto časť obsahovať kapitoly opisujúce, napríklad:

- použitú metodológiu – metodológia experimentu alebo spôsob riešenia úlohy,
- návrh riešenia – celkový opis riešenia a jeho zdôvodnenie,

- implementáciu – najpodstatnejšie implementačné rozhodnutia a ich zdôvodnenie.

Vyhodnotenie

Vyhodnocovacia časť je kľúčovou časťou záverečnej práce. Tato časť obsahuje vyhodnotenie navrhnutého (vytvoreného) riešenia. Uprednostňované je objektívne vyhodnotenie výsledkov práce, ktoré sa opiera o meranie a štatistické metódy, prípadne matematické dôkazy. V prípade nameraných hodnôt musí autor opísať metódu merania, priebeh merania, výsledky a interpretáciu výsledkov v kontexte riešeného problému a stanovených cieľov. Na základe vyhodnotenia riešenia autor opíše prínosy svojej práce. Vyhodnocovacia časť tvorí zvyčajne ¼ jadra práce.

Okrem iného môže byť súčasťou vyhodnotenia:

- zhrnutie dosiahnutých výsledkov,
- interpretácia výsledkov,
- objektívne overenie dosiahnutia cieľov práce

Záver

Záver práce obsahuje zhrnutie výsledkov práce s jasným opisom prínosov a pôvodných (vlastných) výsledkov autora a vyhodnotenie splnenia stanovených cieľov. Je to stručné zhrnutie informácií uvedených v záverečnej práci. Záver by nemal obsahovať nové informácie.

V závere by mal tiež autor poukázať na prípadné otvorené otázky, ktoré sú nad rámec rozsahu práce a mal by odporučiť ďalšie aktivity na pokračovanie pri riešení problému. Rozsah záveru je minimálne 1 celá strana.

Prílohy

Prílohy môžu obsahovať doplňujúce materiály, ktoré sú dôležité pre pochopenie práce, ale nie sú nevyhnutné pre čitateľa, aby mohol pochopiť hlavný obsah práce. Prílohy majú obsahovať aj príručku pre používateľa a systémovú príručku pre nasadenie a pokračovanie vo vývoji.

D LaTeX 101

1 Základy písania

Odseky textu oddeľujte jednoducho prázdnym riadkom. To, že prvý odsek v kapitole nie je odsadený, je štandardný typografický postup a nie je potrebné ho meniť.

Časť textu môžeme *mierne zvýrazniť* pomocou príkazu `\emph{}`. Prípadne použiť **tučné písmo** príkazom `\textbf{}`.

Na vytvorenie zoznamu sa používa prostredie `itemize`:

- raz,
- dva,
- tri.

Zoznam môže byť aj číslovaný ak vymeníme `itemize` za `enumerate`:

1. raz,
2. dva,
3. tri.

2 Členenie textu

Na definovanie kapitol a podkapitol sa používajú príkazy

- `\chapter{}`,
- `\section{}`,
- `\subsection{}`.

Hlbšie úrovne vnorenia sa neodporúča používať. Tak isto neodporúčame mnohonásobne vnorené zoznamy.

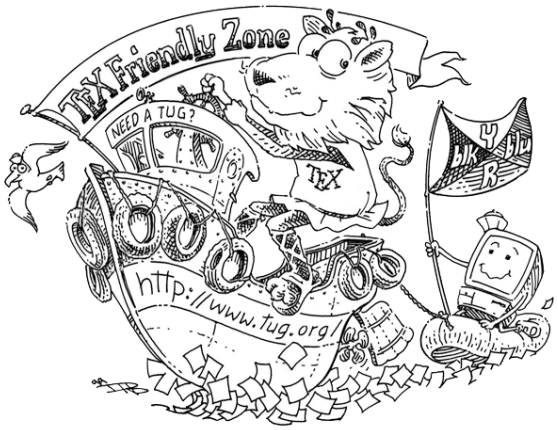
Ak za príkaz pridáte hviezdičku kapitola nebude číslovaná a ani sa nezobrazí v obsahu. Neodporúčame to však používať mimo príloh.

3 Obrázky

Na vkladanie obrázkov sa používa prostredie `figure`:

```
\begin{figure}
  \centering
  \includegraphics[width=0.5\textwidth]{figures/tugboat}
  \caption{\LaTeX{} Friendly Zone \label{o:latex_friendly_zone}}
\end{figure}
```

Výsledkom bude obrázok 1.



Obrázok 1: LaTeX Friendly Zone

Na samotné vloženie obrázka sa používa príkaz `\includegraphics{}`. LaTeX podporuje bežné formáty ako PNG a JPEG. Pre vektorovú grafiku je vhodné použiť formát PDF.

Každý obrázok by mal mať popis, ktorý je uvedený v *caption*. A čo je veľmi dôležité, na každý obrázok by mal byť odkaz v texte. Na to použite príkazy `\label{}` a `\ref{}`. Prvý definuje názov, ktorým sa na obrázok odkazujete, druhý vytvorí odkaz na obrázok. Napríklad, obrázok 1 zobrazuje prostredie priateľské pre používateľov LaTeX-u.

LaTeX má vstavené pravidlá pre umiestnenie obrázkov. Štandardne ich umiestni na vrch stránky, na ktorej sú definované, aby text nebol prerušený. Môžete však použiť voliteľné parametre, aby ste ovplyvnili umiestnenie obrázka. Napríklad `[!ht]` znamená, že obrázok sa má, ak je to možné, umiestniť presne tam, kde je definovaný, inak na vrchu stránky:

```
\begin{figure}[!ht]
```

4 Tabuľky

Tabuľky sa vkladajú do prostredí `table` a `tabular`

Tabuľka 1: Kódy krajín podľa štandardov ISO

Názov krajiny	Alpha 2	Alpha 3	Numeric
Afghanistan	AF	AFG	004
Alandské Ostrovy	AX	ALA	248
Albánsko	AL	ALB	008
Alžírsko	DZ	DZA	012
Americká Samoa	AS	ASM	016
Andorra	AD	AND	020
Angola	AO	AGO	024

Pre sadzbu profesionálne vyzerajúcich tabuliek odporúčame použiť balík *booktabs*[10].

5 Výpisy kódu

Pre výpisy kódu sa používa prostredie `lstlisting`:

Výpis 1: Program, ktorý pozdraví celý svet

```
#include <stdio.h>
int main() {
    /* Print Hello, World! */
    printf("Hello, World!\n");
    return 0;
}
```

Výpisy môžu mať voliteľný nadpis, ktorý sa zobrazí nad výpisom. Tak isto je možné im definovať `label`, na ktorý sa môžete odkazovať (viď výpis 1).

Obsah výpisu môže byť tiež načítaný zo súboru pomocou príkazu:

Výpis 2: Riešenie problému Schody

```
#include <karel.h>

// function definition
void turn_right() {
```

[10] https://en.wikibooks.org/wiki/LaTeX/Tables#Professional_tables

```
    set_step_delay(0);
    turn_left();
    turn_left();
    set_step_delay(400);
    turn_left();
}

int main() {
    turn_on("stairs1.kw");

    set_step_delay(400);

    pick_beeper();
    turn_left();
    step();
    // function call
    turn_right();
    step();

    turn_off();
}
```

6 Citácie

Na citovanie literatúry sa používa balík *biblatex*. Citácie sa vkladajú pomocou príkazu `\cite{}`. Napríklad, [35] je článok, ktorý popisuje štruktúru záverečnej práce.

Bibliografické záznamy sú definované v súbore `bibliography.bib`. Každý záznam má unikátny identifikátor, ktorý sa používa na citovanie. Napríklad, záznam `Beel2010` vyzerá nasledovne:

```
@online{Beel2010,
  title  = {How to write a thesis (Bachelor, Master, or PhD) and ↩
     which software tools to use},
  url    = {https://isg.beel.org/blog/2010/03/02/how-to-write-a-↩
     phd-thesis/},
  author = {Joeran Beel},
  year   = {2010},
  urldate = {2024-09-26}
```

```
}
```

Za znakom `@` je uvedený typ záznamu, v tomto prípade `online`. Iné typy záznamov môžu byť napríklad `article`, `book`, `inproceedings`, a pod. Každý záznam má povinné a nepovinné položky. Pre viac informácií o formáte záznamov v bibliografickom súbore odporúčame pozrieť stránku *The 14 BibTeX entry types*[11].

Pri online zdrojoch nezabudnite uviesť dátum prístupu v položke `urldate`, keďže také zdroje sa môžu časom meniť.

[11] https://www.bibtex.com/e/entry-types/